

Universidad ORT Uruguay

Facultad de Ingeniería

Modelo de Recomendación Para Compras en Supermercado

Entregado como requisito para la obtención del
título de Máster en Big Data

Martín Martínez - 308676

Matias Bernardo - 207870

Florencia Marzano - 292054

Tutor: German Milano

2025

Declaración de Autoría

Nosotros, Matías Bernardo, Martín Martínez y Florencia Marzano, declaramos que el trabajo que se presenta en esta obra es de nuestra propia autoría. Podemos asegurar que:

- La obra fue producida en su totalidad mientras realizamos el Trabajo Final del Máster en Big Data.
- Cuando hemos consultado el trabajo publicado por otros, lo hemos atribuido con claridad.
- Cuando hemos citado obras de otros, hemos indicado las fuentes. Con excepción de estas citas, la obra es enteramente nuestra.
- En la obra, hemos reconocido las ayudas recibidas.
- Cuando la obra se basa en trabajo realizado conjuntamente con otros, hemos explicado claramente qué fue contribuido por otros y qué fue contribuido por nosotros.
- Ninguna parte de este trabajo ha sido publicada previamente a su entrega.



Matías Bernardo

21-04-2025



Martín Martínez

21-04-2025



Florencia Marzano

21-04-2025

Agradecimientos

A nuestras familias y amigos por la contención durante estos años, a nuestro tutor por su apoyo constante y a la Universidad ORT Uruguay.

Abstract

En el comercio electrónico, la capacidad de sugerir artículos se ha convertido en un factor clave para mejorar la satisfacción del cliente y aumentar las ventas. En este contexto, el presente proyecto tiene como objetivo desarrollar un sistema de recomendación aplicado a las compras web de un supermercado, con la finalidad de anticipar las preferencias de compra y personalizar las sugerencias en función del historial de compras de cada cliente.

Con esta finalidad, a partir de un conjunto de datos de 2,019,501 compras en un supermercado, se realizó un análisis exploratorio y se crearon nuevas variables que resultan clave para los siguientes pasos, como la segmentación horaria y la frecuencia de compra. Continuando, con el propósito de adaptar los algoritmos de recomendación a grupos de usuarios con patrones de compra similares, se agruparon a los clientes en cinco perfiles bien diferenciados mediante la aplicación de *K-means*, de forma de mejorar la personalización de las sugerencias. Sobre cada segmento se entrenaron y compararon tres algoritmos de recomendación: factorización matricial con NMF, arquitecturas de “dos torres” con TensorFlow Recommenders y, finalmente, filtrado colaborativo con Surprise.

El modelo optimizado de TensorFlow Recommenders obtuvo un *F1-score* de 0.88, superando al mejor modelo NMF en un 13.6% y al mejor modelo de Surprise en un 18.3%. A su vez, se evidencia la relevancia de técnicas como SMOTE para el balanceo de clases. Estos resultados demuestran que la combinación de la segmentación previa, el aprendizaje profundo y el ajuste fino de hiperparámetros incrementa la precisión y relevancia de las recomendaciones.

Palabras Clave

ETL; Recommender Systems; Machine Learning; Deep Learning; Collaborative Filtering; Content-Based Filtering; Clustering; K-Means; Feature Engineering; One-Hot Encoding; SMOTE; Two-Tower Architecture; Keras Tuner; RMSE; Precision; Recall; F1-score.

Indice

1. Introducción.....	9
1.1 Motivación.....	9
1.2 Estructura del Documento.....	9
2. Marco Teórico.....	11
2.1 Proceso ETL.....	11
2.1.1 Feature Engineering.....	12
2.1.2 One Hot Encoding.....	12
2.2 Clusterización.....	13
2.2.1 PCA (Principal Analysis Components).....	14
2.2.2 K-means.....	15
2.2.3 DB-SCAN.....	16
2.3 Balanceo de Datos.....	17
2.3.1 SMOTE.....	18
2.4 Sistemas de Recomendación.....	19
2.4.1 Modelos y Frameworks Actuales.....	23
2.4.1.1 NMF.....	23
2.4.1.2 TensorFlow Recommenders.....	25
2.4.1.3 Surprise.....	27
2.4.1.4 Implicit.....	28
2.4.1.5 LightFM.....	29
3. Conjunto de Datos.....	30
3.1 Origen.....	30
3.2 Analisis Exploratorio.....	30
3.2.1 Análisis de Ventas por Hora.....	31
3.2.2 Análisis de Ventas por Día de la Semana.....	32

3.2.3 Resumen de Hallazgos.....	35
4. Feature Engineering.....	35
4.1 Transformaciones asociadas al horario y día de la semana.....	36
4.2 Segmentación de Clientes por Cantidad de Órdenes.....	37
4.3 One Hot Encoding.....	38
5. Clusterización.....	39
5.1 Preprocesamiento.....	39
5.2 Implementacion.....	40
5.3. Elección del Número de Clústeres.....	43
5.3.1 Clusterización n=2.....	44
5.3.2 Clusterización n=5.....	46
5.3.3 Clusterización n=5 con Input Reduction.....	49
5.4 Clusterización Resultante.....	54
6. Métricas.....	58
7. Implementación de Algoritmos / Sistemas de Recomendación.....	61
7.1 Balanceo del dataset.....	61
7.2 Modelo NMF.....	63
7.3 Tensor Flow Recommender.....	67
7.3.1 Detalles Técnicos del Modelo.....	68
7.3.2 Resultados.....	70
7.4 Surprise.....	71
7.4.1 Modelo Base.....	72
7.4.2 Exploración de Variantes y Ajuste de Modelos.....	73
7.4.3 Balanceo de base de datos.....	75
7.4.4 Resultados finales.....	75
7.5 Otras Implementaciones.....	76
8. Resultados.....	79
9. Desafíos.....	82

9.1 Desafíos técnicos y de implementación.....	82
9.2 Desafíos prácticos y académicos.....	84
10. Conclusiones y Lecciones Aprendidas.....	86
11. Trabajo Futuro.....	88
11.1 Implementación en una Aplicación Web.....	88
12. Referencias Bibliográficas.....	91
13. Glosario.....	94
Anexos.....	100
Anexo 1: Estructura del repositorio en GitHub.....	101

1. Introducción

1.1 Motivación

En la actualidad, los motores de recomendación forman parte de la experiencia de usuario, tanto para plataformas de streaming como para comercios en línea. Tienen un gran impacto en el comportamiento del consumidor, ya que anticipan intereses, facilitan el descubrimiento de nuevos productos y reducen el tiempo de búsqueda, lo cual contribuye a una mayor satisfacción del consumidor. En el contexto de compras en línea de supermercados, la capacidad de sugerir productos que sean del interés del cliente se traduce en un aumento de productos comprados, maximizando el beneficio del negocio.

Aprovechar la riqueza de los datos los transforma en una ventaja competitiva fundamental para cualquier negocio, ya que permiten comprender los patrones de consumo y generar recomendaciones específicas para cada usuario. En este trabajo se explora la construcción de un sistema de recomendación para compras en línea de un supermercado, combinando técnicas de segmentación de clientes y modelos de *Machine Learning* para anticipar las necesidades de los clientes y mejorar la experiencia de compra, dando como resultado la fidelización de clientes y las ganancias de la empresa.

1.2 Estructura del Documento

El objetivo del presente trabajo es detallar los pasos llevados a cabo para desarrollar un sistema de recomendación para la compras web de un supermercado, iniciando con un conjunto de datos extraídos pero sin preprocesamiento. En el mismo se expondrán los principales tratamientos y exploraciones realizadas, incluyendo hallazgos dificultades y propuestas de mejora y continuidad.

En el Capítulo 2 se explora el Marco Teórico, en el cual se presentan los conceptos principales que respaldan este proyecto, de forma de incorporar las bases teóricas mínimas para comprender el resto del documento.

El documento continúa en el Capítulo 3 en el que se presenta el conjunto de datos, se describen el origen y la estructura de los datos, las variables disponibles y se exploran las transformaciones iniciales realizadas.

El Capítulo 4 explora los aspectos de *Feature Engineering* realizados, detallando la creación de variables temporales y preparando el conjunto de datos para la segmentación y el modelado siguientes.

Adentrándose en los Capítulos 5 y 6 se detalla la Clusterización realizada mediante la aplicación de *K-means*, la implementación del “método del codo” y la validación mediante índices de *Silhouette*, Calinski-Harabasz y Dunn, hasta seleccionar una segmentación óptima de cinco grupos de clientes.

En función de los resultados realizados en el capítulo anterior, en el Capítulo 7 se implementan distintos sistemas de recomendación como el NMF, TensorFlow Recommenders y Surprise para luego llegar al Capítulo 8, en el cual se exponen los resultados y la conclusión del análisis anterior.

Finalmente, el documento culmina en los Capítulos 9, 10 y 11 con los desafíos enfrentados, las principales conclusiones y lecciones aprendidas del trabajo de fin de carrera, así como también una sugerencia de puntos a abordar como trabajo a futuro.

Para facilitar la reproducción y profundización de cada etapa, el código principal se realizó en Python. Los notebooks y los datos intermedios se encuentran disponibles en un repositorio público de GitHub¹. En el Anexo 1 se encuentra un detalle de la estructura del mismo. Es pertinente destacar que para la programación efectuada se utilizaron como apoyo herramientas de inteligencia artificial como ChatGPT, principalmente para las secciones de clusterización y algoritmos de recomendación; por su parte, DALL-E 3 se utilizó para la generación de diagramas e imágenes.

¹ Repositorio del proyecto en Github:
https://github.com/MartinMF88/TrabajoFinal_MasterBigData/tree/main

2. Marco Teórico

Este marco teórico proporciona una base conceptual y contextual para la investigación sobre la implementación de un sistema de recomendación que intente predecir el comportamiento de compradores de un supermercado. Se exploran los fundamentos de los sistemas de recomendación, las metodologías utilizadas en la construcción de los modelos asociados a éstos y su aplicación en diversos contextos, realizando énfasis en la optimización de hiperparámetros y la evaluación del rendimiento entre las diferentes configuraciones testeadas en los diferentes modelos implementados.

2.1 Proceso ETL

El proceso ETL (*Extract, Transform & Load*) es una fase muy importante que se encuentra dentro de la etapa inicial en la construcción de sistemas de recomendación. Una vez definidas las bases de datos, es necesario extraer la información y transformarla para que sea más clara y útil al momento de aplicar algoritmos de clusterización o sistemas de recomendación. Luego de realizados los procesos de transformación, se procede a cargar el conjunto de datos dejándolo disponible para el uso en los diferentes modelos.

- **Etapas de extracción:** en esta etapa se recopilan datos de diversas fuentes, como pueden ser bases de datos transaccionales, archivos CSV (*Comma-Separated Values*) o APIs (*Application Programming Interfaces*) externas.
- **Etapas de transformación:** se aplican diversas técnicas de “limpieza” como ser eliminación de outliers, datos duplicados, etc, así como también normalización y enriquecimiento de datos. Esto permite asegurar la calidad y coherencia de la información a utilizar.
- **Etapas de carga:** en esta etapa los datos procesados se almacenan en bases de datos estructuradas o sistemas distribuidos de almacenamiento (*data warehouse, data lake o data lake-house*). Esto facilita el acceso eficiente a la información para entrenar modelos de recomendación y realizar análisis exploratorios.

2.1.1 Feature Engineering

Dentro de la etapa de Transformación del proceso ETL, encontramos el proceso de FE (*Feature Engineering*), la cual es una fase necesaria en el desarrollo de sistemas de recomendación. Permite transformar los datos en representaciones de mayor utilidad para mejorar el desempeño de los modelos. En este contexto, “características” (*features*) son las variables que describen los datos y que el modelo usa para aprender y hacer predicciones. Luego de que los datos fueron procesados de manera de asegurar una información limpia y estructurada, se procede a identificar cuales son las características que pueden aportar mayor valor predictivo y cómo construirlas y relacionarlas de manera eficiente.

Dentro del proceso de FE se pueden identificar varias etapas que se detallan a continuación:

- **Selección de *features*:** en esta etapa se procede a identificar cuales son las variables más relevantes para el modelo, se descartan las que no aportan información útil o pueden generar ruido en el entrenamiento.
- **Creación de nuevas características:** en esta etapa se generan atributos contruidos a partir de cálculos o relaciones entre los datos originales, ejemplos de esto pueden ser estadísticas agregadas, combinaciones de dos o más variables o representaciones categóricas mejoradas.
- **Transformación de características:** en esta etapa se aplican diversas técnicas como normalización, estandarización, escalado y codificación de variables categóricas, esto se realiza con el fin de asegurar que los datos sean adecuados para el tipo de algoritmo utilizado.

2.1.2 One Hot Encoding

Dentro del proceso de *Feature Engineering*, se encuentra la técnica de *One Hot Encoding*. Esta es una técnica que se utiliza para convertir variables categóricas en una

representación numérica que sea interpretable para modelos de ML (*Machine Learning*). Consiste en transformar cada categoría en un vector binario donde solo una posición toma el valor de 1 y las demás se mantienen en 0.

One Hot Encoding es útil en sistemas de recomendación porque permite transformar variables categóricas en una representación numérica que los algoritmos pueden interpretar de forma más eficiente. En modelos basados en contenido, esta codificación facilita la comparación entre usuarios y productos al convertir atributos como la categoría de un ítem o la ubicación de un usuario en vectores binarios. Esto mejora la forma en que el sistema calcula similitudes y, en consecuencia, contribuye a generar recomendaciones más precisas y relevantes.

2.2 Clusterización

El proceso de clusterización es una fase que aporta ciertas ventajas al desarrollo de sistemas de recomendación. Mediante esta técnica se agrupan usuarios o productos con características similares, generando de esta forma conjuntos homogéneos de datos que comparten ciertas características entre sí. Esto reduce la complejidad del sistema mediante la agrupación de conjuntos de datos similares, lo que facilita la personalización de las recomendaciones. Además es útil en casos de sistemas con grandes volúmenes de información, ya que permite trabajar con subconjuntos de datos más pequeños.

Para avanzar con la técnica de clusterización se deben seguir ciertos pasos para asegurar una segmentación de datos efectiva y útil para los sistemas de recomendación a desarrollar. Las etapas son enumeradas a continuación:

- **Selección de variables:** en esta etapa se procede a seleccionar cuales son las características más representativas que permitan diferenciar grupos de usuarios o productos de manera eficiente.
- **Aplicación de algoritmos de clusterización:** en esta etapa se utilizan diferentes técnicas como por ejemplo *K-means* o DBSCAN (*Density-Based Spatial Clustering of Applications with Noise*) para formar los diferentes grupos, la

elección de cada algoritmo va a depender de la naturaleza de los datos y el objetivo para el cual se realizará la clusterización.

- **Evaluación y validación de clústeres:** es la etapa final en la cual se utilizan métricas como el índice de Calinski-Harabasz, el coeficiente de silueta o la distancia intra e inter-clúster para determinar la calidad de la segmentación obtenida.

2.2.1 PCA (Principal Analysis Components)

El PCA es una técnica estadística utilizada dentro del proceso de transformación de datos. Permite reducir la dimensionalidad de un conjunto de datos, manteniendo la mayor cantidad de información posible. Al proyectar los datos en un nuevo espacio formado por componentes principales, se facilita el análisis y mejora el rendimiento de los algoritmos utilizados posteriormente, como los de clusterización o sistemas de recomendación.

En contextos donde existen múltiples variables que pueden estar correlacionadas entre sí, PCA resulta muy efectivo para identificar patrones y eliminar redundancias. El proceso de PCA cuenta con varias etapas que se detallan a continuación:

- **Estandarización de los datos:** dado que PCA se basa en la varianza de los datos, es necesario que todas las variables estén en la misma escala. Para cumplir con esto, se aplica una normalización o estandarización previa.
- **Cálculo de la matriz de covarianza:** una vez que los datos fueron estandarizados, se procede a calcular la matriz de covarianza, este procedimiento permite identificar cómo se diferencian las variables entre sí.
- **Obtención de valores y vectores propios:** se identifican nuevas direcciones o combinaciones de las variables originales que capturan la mayor variabilidad posible en los datos. Estas nuevas combinaciones son ordenadas según la cantidad de información que explican. Las primeras direcciones seleccionadas representan las que mejor resumen el comportamiento general del conjunto de

datos.

- **Selección de componentes principales:** se seleccionan los componentes que explican mayor proporción de la varianza total. Esta selección puede estar guiada por criterios como el *método del codo* en la gráfica de varianza.
- **Transformación de los datos:** en la última etapa del proceso de PCA, los datos originales se reorganizan utilizando las combinaciones seleccionadas previamente, esto genera un nuevo conjunto de variables no correlacionadas [1].

2.2.2 K-means

K-means es un algoritmo de aprendizaje no supervisado utilizado en técnicas de clusterización para agrupar datos en K números de conjuntos distintos. Su funcionamiento consiste en buscar la minimización de la variabilidad dentro de cada clúster, lo que permite identificar patrones en grandes volúmenes de datos [1].

Minimización de la variabilidad:

Mediante un proceso iterativo, *K-means* funciona siguiendo los pasos detallados a continuación:

1. **Inicialización:** se eligen de forma aleatoria K puntos en el espacio de los datos que van a actuar como centroides iniciales de los clústeres. Entendiendo por centroide, al punto que representa el centro de un clúster, que se calcula como el promedio de todas las observaciones que pertenecen a ese grupo.
2. **Asignación de puntos a clústeres:** cada punto del conjunto de datos se asigna al clúster cuyo centroide esté más cerca, para la medición generalmente se utiliza la distancia euclidiana, que se calcula como la raíz cuadrada de la suma de las diferencias al cuadrado entre las coordenadas de los puntos.

Si tenemos dos puntos $A (x_1, y_1)$ y $B (x_2, y_2)$, la distancia euclidiana es:

$$d(A, B) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2} \quad (2.1)$$

3. **Cálculo de centroides:** una vez que todos los puntos fueron asignados a un clúster, se procede a recalcular la posición de cada centroide como el promedio de todos los puntos dentro de cada clúster.
4. **Repetición hasta convergencia:** se procede a repetir los pasos 2 y 3 hasta que los centroides dejan de cambiar de manera significativa. Esto implica que los puntos ya no se mueven de clúster, y que la variabilidad de cada clúster se reduce al mínimo valor posible.

2.2.3 DB-SCAN

Se trata de un algoritmo de aprendizaje no supervisado, utilizado en técnicas de clusterización basado en densidad. La principal diferencia con *K-means* es que DBSCAN no requiere especificar el número de clústeres previamente, sino que este algoritmo identifica regiones de datos densamente pobladas y las agrupa en clústeres diferentes, esto permite detectar relaciones complejas en los datos y manejar eficientemente valores atípicos [2].

DB-SCAN funciona de la siguiente manera:

1. Selección de parámetros, en primer lugar se establecen los parámetros de ejecución del algoritmo.
 - a. **Epsilon (ϵ):** es la medida del radio para determinar si un punto es vecino de otro punto.
 - b. **MinPts:** es el número mínimo de puntos dentro del radio ϵ para considerar a un punto como punto central del clúster.

2. Clasificación de puntos:
 - a. **Punto central:** debe tener al menos MinPts dentro de su radio (ϵ) para ser considerado como tal.
 - b. **Punto de frontera:** es el punto que no tiene suficientes vecinos para ser considerado punto central, pero está dentro del radio (ϵ) de otro punto central.
 - c. **Punto ruido:** no cumple ninguna de las condiciones anteriores, por lo cual no forma parte de ningún clúster
3. **Expansión de clústeres:** A partir de un punto central se agregan todos los puntos de frontera, formándose así los diferentes clústeres.
4. **Finalización:** Se repite el proceso hasta clasificar todos los puntos en clústeres o identificarlos como ruido.

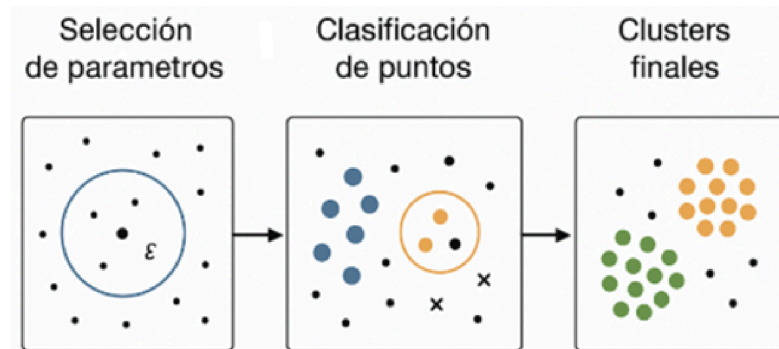


Figura 2.1 Proceso de agrupamiento con DBSCAN: selección de parámetros, clasificación de puntos y formación de clústeres [3].

2.3 Balanceo de Datos

Como se ha mencionado en secciones anteriores, en el contexto del desarrollo de modelos de aprendizaje automático y sistemas de recomendación, la calidad de los datos cobra importancia.

Uno de los problemas más comunes ante esta situación, es el desbalance de clases en la variable a predecir, el cual ocurre cuando una o varias categorías tienen una proporción de observaciones mucho más elevada que el resto. Este inconveniente lleva a que se puedan generar modelos sesgados que se orienten más a realizar predicciones que favorezcan más a las categorías mayoritarias, desfavoreciendo a las clases minoritarias.

2.3.1 SMOTE

Es una técnica de sobremuestreo sintético basada en la generación de nuevos ejemplos mediante interpolación. En lugar de simplemente replicar datos existentes de la clase minoritaria, SMOTE crea nuevos puntos en el espacio de características tomando pares de observaciones cercanas. Sigue los siguientes pasos:

1. **Seleccionar una instancia minoritaria aleatoria:** comienza tomando un punto de la clase minoritaria.
2. **Buscar sus vecinos más cercanos:** luego se identifican los k puntos más cercanos en la misma clase usando una métrica de distancia (generalmente se utiliza la distancia euclidiana).
3. **Interpolación:** finalmente se genera un nuevo punto sintético que toma un punto original y lo desplaza en la dirección de uno de sus vecinos más cercanos, para realizar este cálculo se aplica la ecuación detallada a continuación

$$X_{nuevo} = X_i + \lambda (X_j - X_i) \quad (2.2)$$

Donde X_i es el punto original, X_j uno de sus vecinos más cercanos, y λ un valor aleatorio en un rango $[0,1]$.

4. **Repetición del proceso:** en la etapa final se procede a generar tantas muestras sintéticas como sean necesarias hasta alcanzar el balance óptimo

2.4 Sistemas de Recomendación

En la actualidad, los sistemas de recomendación han tomado gran relevancia en la era digital, aportan valor en la personalización de contenido de diversas industrias como lo son el comercio electrónico (tiendas como Amazon, E-bay, etc), el entretenimiento (plataformas de streaming de contenidos) y los servicios financieros (sugerencias de seguros personalizados o tarjetas de crédito). Estos sistemas utilizan algoritmos avanzados que permiten predecir las preferencias de los usuarios y así proporcionar sugerencias personalizadas que permiten mejorar la experiencia del usuario y maximizar el valor para las empresas.

Caso Netflix:

Un caso representativo es el sistema de recomendación de Netflix [4]. Este sistema utiliza un enfoque basado en filtrado colaborativo y técnicas avanzadas de ML para mejorar la precisión de las sugerencias que se generan para los usuarios mientras navegan en la plataforma. Netflix a lo largo de los años ha optimizado sus algoritmos de manera muy eficiente, integrando metodologías como SVD (*Singular Value Decomposition*) y *deep learning* para personalizar las recomendaciones según los hábitos de visualización y preferencias de los diferentes usuarios.

Existen diferentes enfoques al momento de diseñar un sistema de recomendación. Entre éstos se destacan los modelos basados en filtrado colaborativo, filtrado basado en contenido y enfoques híbridos. El filtrado colaborativo, puntualmente el basado en SVD , ha demostrado ser efectivo en la predicción de preferencias al identificar patrones en los datos de interacciones usuario-producto. Como se mencionaba previamente, el caso Netflix es un claro ejemplo de implementación de este tipo de sistemas, arrojando resultados que respaldan este buen rendimiento mencionado.

En el ámbito del comercio, y en particular en los supermercados que es el rubro sobre el cual se desarrolla el presente trabajo, los sistemas de recomendación han demostrado ser herramientas muy útiles para mejorar la experiencia de los clientes y, a su vez, optimizar las estrategias comerciales para adaptar la toma de decisiones a las preferencias de los

clientes. La capacidad que tienen para predecir las preferencias de compra de los consumidores, ha permitido a las empresas ofrecer recomendaciones personalizadas, lo cual ayuda a lograr la fidelización de los compradores lo que por consiguiente genera aumento de las ventas.

A medida que los sistemas de recomendación han evolucionado, se han desarrollado nuevas herramientas que han permitido mejorar el desempeño y a su vez han facilitado su implementación. En este contexto, han aparecido una gran variedad de bibliotecas y *frameworks* para facilitar la construcción y optimización de modelos de recomendación, lo cual ha permitido a los desarrolladores e investigadores centrarse en la mejora del rendimiento y la personalización de las predicciones.

Enfoques usuales:

Existen diferentes enfoques de sistemas de recomendación, siendo los principales CBF (*Content Based Filtering*) y CF (*Collaborative Filtering*). En la actualidad, ambas metodologías suelen combinarse para formar sistemas de recomendación híbridos.

CBF recomienda elementos similares a los que el usuario ha elegido en el pasado, basándose en el contenido de los productos.

En los CBF, el motor de recomendaciones compara el perfil del usuario con el perfil del ítem para predecir qué tanto le podría interesar ese ítem al usuario y de esta forma generar las recomendaciones.

- **Perfil del ítem:** Es la forma en que el sistema representa cada producto o contenido. Está compuesto por sus características (atributos), como pueden ser el género de una película, el año de lanzamiento, el director, entre otros datos descriptivos. Toda esta información ayuda a definir de qué trata el contenido.
- **Perfil del usuario:** Refleja los intereses y preferencias del usuario, y se construye en base a su historial de interacción con el sistema. Incluye lo que el usuario ha calificado, buscado, o marcado como favorito. A partir de esos datos, el sistema busca interpretar qué tipo de contenido prefiere [5].

Funcionamiento de CBF:

Este enfoque crea una matriz que representa las características de cada ítem y las preferencias del usuario, basándose en su historial de interacciones con el sistema. Luego, se comparan vectores de características mediante el uso de métricas de similitud para determinar qué ítems son más afines a las preferencias del usuario.

Una vez definidos los vectores, se aplican algoritmos de comparación entre el perfil del usuario y el perfil de los ítems disponibles para estimar el grado de interés que el usuario podría tener en los productos.

Las métricas más comunes utilizadas en CBF para calcular similitudes son: similitud del coseno y distancia euclidiana.

La similitud del coseno es una forma de medir qué tan parecidos son dos vectores, observando el ángulo que existe entre ellos. En el contexto de sistemas de recomendación, estos vectores pueden representar las calificaciones que un usuario dio a ciertos productos, o las características de distintos ítems. Cuanto más pequeño sea el ángulo entre los vectores, más similares son. Esta similitud se expresa con un valor entre -1 y 1:

- 1 significa que los vectores son similares.
- 0 significa que no tienen relación.
- -1 indica que son totalmente opuestos.

Por otro lado, la distancia euclidiana mide la distancia directa entre dos vectores en el espacio. En el contexto de sistemas basados en CBF, se usa para determinar cuán diferentes son dos productos teniendo en cuenta todas sus características. A menor distancia, mayor similitud entre los ítems. A diferencia del coseno, que se enfoca en la orientación, la distancia euclidiana se enfoca en la magnitud de las diferencias, por lo que puede ser sensible a la escala de los datos.

Ambas métricas permiten al sistema identificar productos similares a aquellos que el usuario ya ha valorado positivamente, y así generar recomendaciones personalizadas en base al contenido.

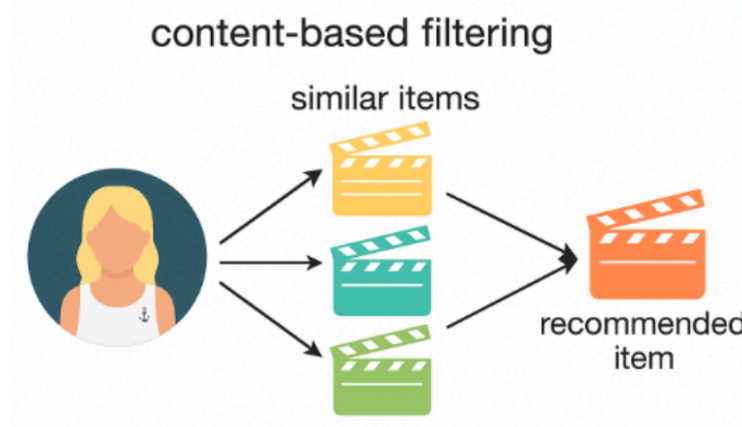


Figura 2.2 Filtrado basado en contenido [3].

CF propone recomendaciones basadas en las preferencias de usuarios con características similares entre sí. Relaciona a los individuos mediante preferencias, asociando gustos similares de las personas para agruparlas en grupos y recomendarles en conjunto.

Funcionamiento de CF:

Este método utiliza una matriz para mapear el comportamiento de los usuarios asociado a cada elemento consumido por el mismo. Luego, el sistema extrae valores de la matriz para representarlos como puntos de datos en un espacio vectorial. Finalmente, se aplican diferentes métricas para medir la distancia entre puntos con el objetivo de determinar la similitud entre usuario-usuario y artículo-artículo.

Dentro de las métricas más relevantes utilizadas para determinar similitud en CF se encuentran: similitud de coseno y PCC (coeficiente de correlación de Pearson).

PCC se usa para medir qué tan similares son dos usuarios o productos, comparando cómo califican los mismos ítems. Básicamente, calcula si existe una relación lineal entre las puntuaciones que dan dos usuarios o reciben dos productos.

El resultado también se encuentra en el intervalo de -1 a 1 [6]:

- 1 indica una correlación perfecta positiva,
- 0 indica ninguna correlación,
- -1 es una correlación inversa.

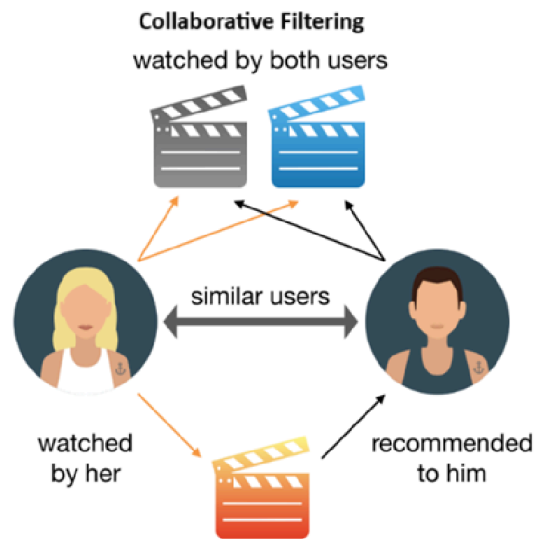


Figura 2.3 Filtrado Colaborativo [7].

2.4.1 Modelos y Frameworks Actuales

2.4.1.1 NMF

NMF es un método de descomposición matricial utilizado en sistemas de recomendación basados en filtrado colaborativo. Este método permite descomponer una matriz de interacciones entre usuarios y productos en dos matrices de menor dimensión, garantizando que todos los valores sean no negativos. Esto no solo facilita la interpretación de los resultados, sino que también permite captar atributos latentes relacionados a las preferencias ocultas de usuarios, mejorando así la precisión de las predicciones. Los atributos latentes son la representación de vectores de características creados por cada usuario y cada producto.

Características clave de NMF:

- **Descomposición sin valores negativos:** A diferencia de otros métodos de descomposición, NMF trabaja únicamente con valores positivos. Esto permite que los factores latentes resultantes (que representan características implícitas de usuarios y productos), sean fácilmente interpretables. En este contexto, los factores latentes pueden entenderse como dimensiones ocultas que explican el comportamiento de compra de los clientes.
- **Eficiencia en contextos de datos dispersos:** A diferencia de otros métodos, NMF muestra un gran rendimiento en escenarios donde la matriz usuario-producto es escasa, es decir, contiene muchas celdas vacías debido a la falta de información.
A pesar de esta limitación, NMF es capaz de extraer información valiosa y generar recomendaciones, incluso cuando existen pocas interacciones.
- **Optimización y adaptabilidad:** El modelo puede ser entrenado utilizando algoritmos como MUR (*Multiplicative Update Rule*) o ALS (*Alternating Least Squares*), los cuales permiten ajustar sus parámetros para optimizar el rendimiento del sistema. Además, NMF es altamente adaptable y puede integrarse con otras técnicas para potenciar su capacidad predictiva [8] [9].

Fundamento Matematico:

El objetivo de NMF es descomponer una matriz $V \in R_{m \times n}$ de elementos no negativos, en el producto aproximado de dos matrices de dimensiones más pequeñas cuyos elementos también son no negativos, denominadas $W \in R_{m \times k}$ y $H \in R_{k \times n}$ de tal forma que $V \approx W \times H$. La matriz V representa las interacciones originales entre los usuarios y productos, W contiene las características latentes de usuarios, mientras que H representa las características latentes de los productos. El parámetro k indica la cantidad de factores latentes utilizados en la composición de las matrices W y H . El mismo determina el nivel de granularidad, por lo que es necesario definirlo previo a la ejecución del modelo.

A modo de ejemplo, si un usuario tiene preferencia por productos frescos en lugar de panadería, los productos frescos tendrán un peso mayor que los de panadería. Para predecir la preferencia de un producto por usuario, se comparan estos vectores mediante un producto escalar y como todos los valores son positivos, los factores siempre suman a la predicción.

2.4.1.2 TensorFlow Recommenders

Otro ejemplo es el uso de TFRS (*TensorFlow Recommenders*)[10], una biblioteca diseñada específicamente para la implementación de sistemas de recomendación basados en aprendizaje profundo. TFRS permite la construcción de modelos altamente personalizables, combinando técnicas de *embeddings* (vectores numéricos que representan a usuarios o productos y capturan similitudes entre ellos), redes neuronales y optimización avanzada. La optimización de los hiperparámetros en estos modelos, como la dimensionalidad de los *embeddings*, la tasa de aprendizaje y la regularización, es crucial para mejorar su desempeño y garantizar recomendaciones precisas y relevantes. Gracias a su integración con el ecosistema de TensorFlow, TFRS facilita la experimentación y escalabilidad en la implementación de sistemas de recomendación modernos.

Algo sumamente útil a destacar en TFRS es que facilita la implementación de modelos híbridos, ya que permite combinar técnicas de filtrado colaborativo y contenido.

Características clave de TFRS:

- **Arquitectura basada en Aprendizaje Profundo:** soporta modelos avanzados de *deep learning*, como factorización matricial, modelos de clasificación y aprendizaje de *embeddings*. Por otro lado facilita la implementación de técnicas modernas como Two-Tower Models, que consiste en combinar información de usuarios y productos para mejorar la precisión de las predicciones (sistema de recomendación híbrido).
- **Fácil Implementación y Evaluación:** Está integrado con TensorFlow y es compatible con herramientas como la biblioteca Keras, esto entre otros

beneficios hace más fácil la construcción y entrenamiento de modelos.

- **Personalizable y Extensible:** Los usuarios pueden crear arquitecturas específicas según sus necesidades y adaptar estos modelos en base a cambios que puedan surgir sobre el proceso de implementación.
- **Eficiencia en Grandes Volúmenes de Datos:** Puede manejar una gran cantidad de interacciones de usuario-producto, ya que optimiza el rendimiento mediante el uso de técnicas de procesamiento distribuido [11].

Two Towers Model:

El modelo de “dos torres” (*Two Towers Model*), plantea una técnica donde cada “torre” es una sub red neuronal completa e independiente que aprende un *embedding*; una se dedica a procesar los identificadores y características de los usuarios mientras que la otra torre realiza el mismo procedimiento con los diferentes productos. Cada torre se implementa por separado y se combinan al final para predecir la afinidad entre clientes y productos. Esta estructura permite capturar de forma eficiente las relaciones latentes entre usuarios y artículos.

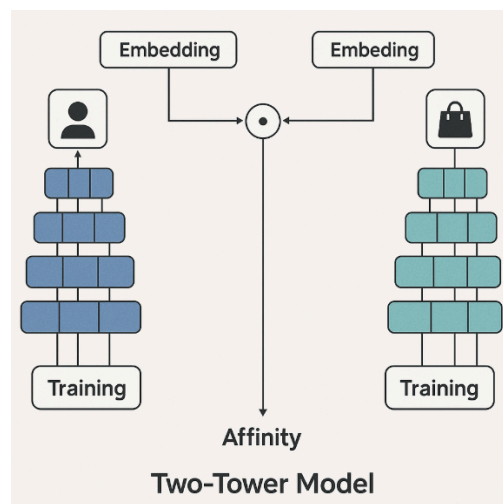


Figura 2.4 Modelo de dos Torres [3].

2.4.1.3 Surprise

En este contexto, *Simple Python Recommendation System Engine* (también llamada Surprise)[12] es una de las bibliotecas utilizadas en la actualidad para la implementación de sistemas de recomendación, la cual ha sido implementada en el presente trabajo. La misma proporciona algoritmos para la predicción de preferencias en conjuntos de datos de usuario-producto. Dado que esta biblioteca ha sido desarrollada para el modelado de preferencias en datos del tipo usuario-producto, en el caso de estudio, ha sido de gran utilidad para encontrar patrones de preferencia entre los compradores sobre los diferentes ítems que ofrece el supermercado.

Características Clave de Surprise:

- **Diversidad de Algoritmos Integrados:** Soporta una gran variedad de técnicas de recomendación, entre las cuales se incluyen SVD, KNN (*K Nearest Neighbors*), SVD++, NMF (*Non-negative Matrix Factorization*) y otros métodos que se basan en la similitud de usuarios o ítems.
- **Fácil Implementación y Evaluación:** posee métricas integradas como *RMSE*, *MAE* y *Precision* en *top-N* recomendaciones, lo cual facilita la evaluación del rendimiento del modelo. Además, Surprise proporciona herramientas para evaluar la precisión de los algoritmos utilizando técnicas como la validación cruzada.
- **Personalizable:** Los usuarios pueden crear algoritmos personalizados y extender la funcionalidad de la biblioteca.
- **Eficiencia en Datos Escasos:** funciona bien en situaciones en las que el conjunto de datos se encuentra demasiado disperso o incompleto. Además, algo sumamente útil en Surprise, es que implementa técnicas como la descomposición en valores singulares para extraer relaciones latentes entre los

usuarios y los productos, incluso cuando existen pocos datos disponibles.

- **Herramientas de Evaluación Eficientes:** Surprise proporciona herramientas para evaluar la precisión de los algoritmos utilizando técnicas como la validación cruzada [13].

2.4.1.4 Implicit

Implicit[14] es una librería optimizada para realizar recomendaciones utilizando filtrado colaborativo basado en factores latentes (características ocultas que existen entre las relaciones de los usuarios). A diferencia de otros métodos que trabajan con relaciones explícitas, Implicit está enfocado específicamente en analizar interacciones implícitas, como pueden ser clics, compras, visualizaciones de productos o tiempo en pantalla, sin necesidad de que los usuarios realicen puntuaciones directas.

Características clave de Implicit:

- **Especializado en Datos Implícitos:** Enfoca su performance en interacciones que no poseen calificaciones explícitas de los usuarios, sino que utiliza señales indirectas como tiempo de visualización o clics en productos.
- **Optimización y Eficiencia:** Está diseñado para gestionar grandes volúmenes de datos dispersos, además optimiza los cálculos aplicando técnicas como multiprocessing y procesamiento en GPU.
- **Personalización y Flexibilidad:** Permite realizar el cambio o ajuste de diferentes hiperparámetros como por ejemplo el número de factores latentes, la regularización o la tasa de aprendizaje. Además se puede combinar con otras técnicas de filtrado colaborativo así como también con modelos de aprendizaje automático que permitan mejorar la precisión en distintos contextos o tipos de datos [15] [16].

2.4.1.5 LightFM

LightFM[17] es una biblioteca optimizada para la creación de sistemas de recomendación híbridos que combinan filtrado colaborativo y filtrado basado en contenido. Permite generar recomendaciones personalizadas aprovechando tanto interacciones realizadas por los usuarios como también información adicional sobre productos y los propios usuarios.

Características clave de LightFM:

- **Modelo Híbrido:** Implementa la técnica de filtrado colaborativo combinada con filtrado basado en contenido, lo cual mejora la calidad de las recomendaciones aun cuando existe poca información sobre algunos elementos (problema de "cold start").
- **Optimización mediante Aprendizaje Automático:** Utiliza técnicas como factorización matricial con *embeddings*, que permite encontrar relaciones complejas en los datos. A su vez, incorpora funciones de pérdida como log-loss y BPR (Bayesian Personalized Ranking) que son útiles para optimizar la personalización de las recomendaciones.
- **Eficiencia y Escalabilidad:** Soporta el entrenamiento en GPU, lo que es especialmente útil para acelerar el procesamiento en casos de sistemas con grandes volúmenes de datos [18].

3. Conjunto de Datos

En este capítulo se describirán los datos utilizados, así como su transformación y procesamiento. Se detalla el origen de los datos, su formato y las modificaciones necesarias para su correcto manejo y análisis. Además, se explicarán las transformaciones aplicadas para garantizar la coherencia y calidad de la información.

3.1 Origen

Para este trabajo se decidió utilizar un conjunto de datos libre proveniente de la web Kaggle². Este conjunto de datos contiene las ventas de un supermercado en EE.UU. Este conjunto, proporciona detalladamente las órdenes de compra realizadas por los distintos clientes, esto permite examinar el comportamiento de consumo y la dinámica de adquisición de productos. Se encuentra comprimido en formato CSV y está compuesto por múltiples variables que describen tanto las órdenes de compra como los productos adquiridos, agrupados por el ID del cliente.

3.2 Analisis Exploratorio

Antes de comenzar con la ingeniería de características, se llevó a cabo un análisis exploratorio de los datos (EDA por sus siglas en inglés) orientado a comprender la estructura y las características principales del conjunto de datos. Se analizaron las franjas horarias en las que se concentra el mayor volumen de transacciones y que días de la semana presentan picos de actividad, con el fin de identificar hábitos de compra cotidianos. Asimismo, se analizó cómo varía la proporción de recompra (reordered) a lo largo del tiempo y si la combinación entre el día y la hora revela comportamientos diferenciados. Estos hallazgos permiten definir y validar las variables temporales para luego emplearlas en los modelos de clusterización y recomendación.

Como se mencionó anteriormente, el conjunto de datos obtenido de Kaggle se encuentra comprimido en formato CSV. Para comenzar con el EDA, se carga el archivo comprimido

² Dataset utilizado:

<https://www.kaggle.com/datasets/hunter0007/ecommerce-dataset-for-predictive-marketing-2023>

Supermercado.zip, extrayendo el archivo Supermercado.csv. Para esto, se utilizó la biblioteca Pandas³, que también se utilizó en el primer acercamiento a los datos.

El conjunto de datos (en adelante, df), contiene aproximadamente 2 millones de filas y 12 columnas. Cada fila representa el detalle de una compra realizada por un usuario. A continuación, se presenta un desglose de las columnas que componen el df, junto con una breve descripción de cada una:

Variable	Descripción
order_id	Identificador único de cada orden de compra.
user_id	Identificador único de cada cliente.
order_number	Número secuencial de la orden realizada por el cliente.
order_dow	Día de la semana en el que se efectuó la compra
order_hour_of_day	Hora del día en la que se realizó la compra.
days_since_prior_order	Días transcurridos desde la compra anterior del mismo cliente.
product_id	Identificador único del producto
add_to_cart_order	Posición del producto dentro del carrito de compras
reordered	Indicador binario que señala si el producto fue comprado nuevamente por el mismo cliente
department_id	Identificador único del departamento
department	Nombre del departamento.
product_name	Nombre del producto.

Tabla 3.1 Descripción de las variables contenidas en el conjunto de datos.

3.2.1 Análisis de Ventas por Hora

Para comenzar a analizar el comportamiento de compra de los clientes, se decide evaluar la cantidad de compras efectuadas por hora y por día. Se identificó que la mayor cantidad de compras se concentran entre las 9:00 y las 16:00 horas, siendo las 10:00 el horario con más transacciones.

³ Documentación de Pandas:
<https://pandas.pydata.org/>

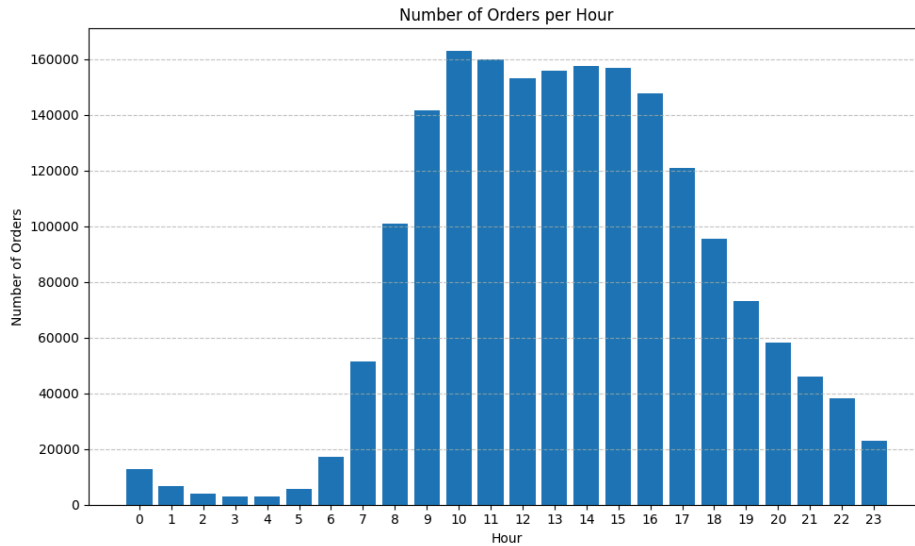


Figura 3.1: Gráfico de la cantidad de órdenes que se realizan según la hora del día.

3.2.2 Análisis de Ventas por Día de la Semana

Por otro lado, se busca comprender qué día de la semana se efectúan más compras por parte de los clientes. Se detectó que lunes y martes presentan la mayor concentración de órdenes, aproximadamente un tercio, lo cual puede insinuar preferencia a realizar las compras los primeros días de la semana.

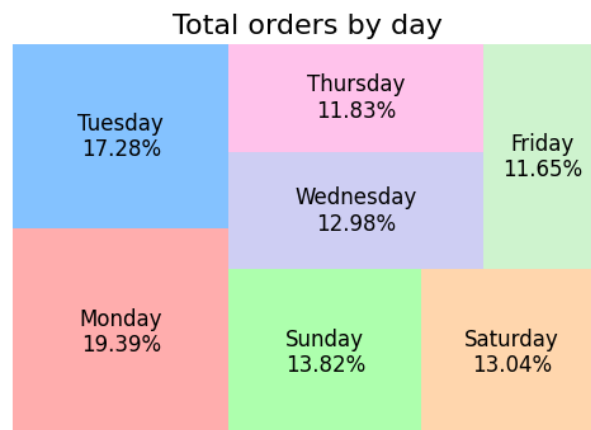


Figura 3.2: Distribución de pedidos por día de la semana.

Con el objetivo de facilitar el análisis, se decidió realizar una segmentación de horarios por momento del día, denominándose "Morning", "Afternoon", "Night" y "Dawn" :

Horarios	Momento del Dia
6:00 - 12:00	Morning
13:00 - 17:00	Afternoon
18:00 - 22:00	Night
23:00 - 5:00	Dawn

Tabla 3.2: Segmentación de horarios según el momento del día.

Lo que se busca mediante esta transformación, es agrupar a las compras de los usuarios por etapas más acotadas del día, buscando de esta forma encontrar comportamientos ocultos, asociados a compras, realizadas por ejemplo, para consumo nocturno que pueda tener carácter recreativo.

Con esta agrupación, se encuentra que la gran mayoría de órdenes son realizadas en el transcurso de la mañana hasta la tarde:

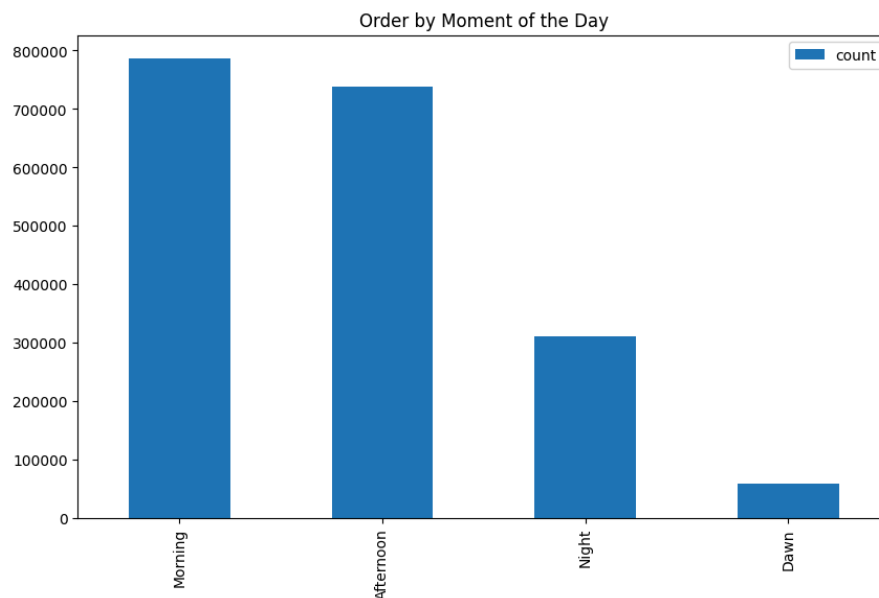


Figura 3.3: Cantidad de órdenes por momento del día.

En el siguiente paso, se busca unir los últimos dos análisis, es decir, se busca ver si el comportamiento de compra por momento del día se mantiene a lo largo de la semana:

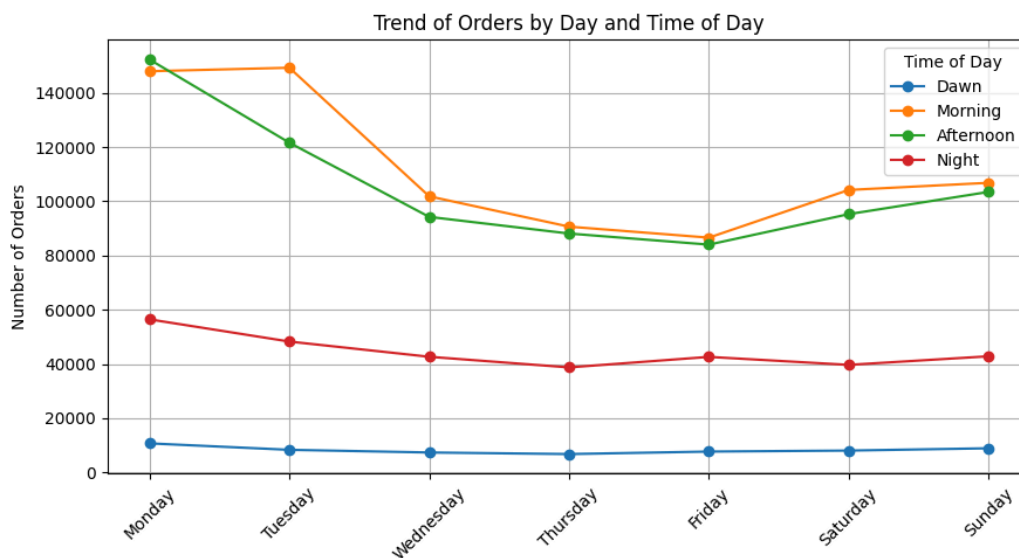


Figura 3.4: Tendencia de órdenes por día y franja horaria.

Se detecta que las tendencias de compra en la noche y la madrugada son relativamente estables, presentando una caída en el volumen de demanda desde el lunes hasta el jueves. Por otro lado, los horarios de la mañana muestran un crecimiento de las compras desde el viernes que se mantiene hasta el martes, donde baja abruptamente estabilizandose hasta el viernes. En el caso de la tarde, se detecta que este fenómeno es similar, con la diferencia de que el crecimiento en el volumen de compra se detiene el día lunes, donde baja abruptamente hasta el viernes.

En la siguiente gráfica, se puede ver claramente que la mayor cantidad de órdenes se realizan entre la mañana y la tarde. El nivel de compras de los lunes y martes, como se mencionó anteriormente, es superior al nivel de órdenes el resto de los días de la semana. Se detecta la particularidad de que los martes existe una caída significativa en el volumen de pedidos por las tardes, a diferencia de en el resto de los días.

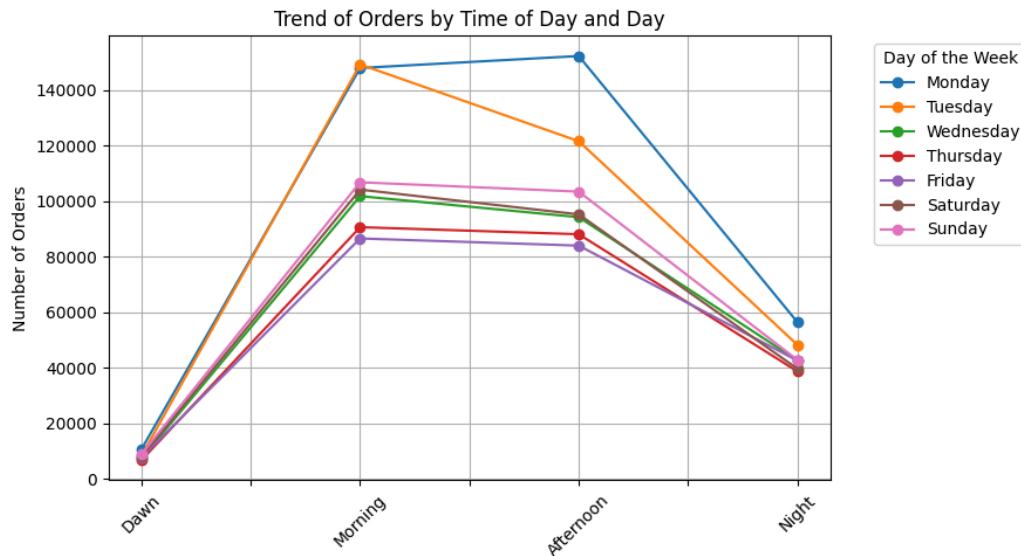


Figura 3.5 Tendencia de órdenes por día y momento del día.

Posteriormente, se efectuó un análisis descriptivo detallado que permitió identificar patrones generales, tales como los horarios y días de mayor actividad, las combinaciones de productos más frecuentes y la frecuencia de recompra, entre otros. Estos hallazgos brindaron una primera aproximación al comportamiento de los consumidores.

3.2.3 Resumen de Hallazgos

El análisis exploratorio reveló que la mayor parte de las compras se realizan entre las 9:00hs y las 16:00hs, concentrándose especialmente a las 10:00hs. En cuanto al día de la semana, se observó que los Lunes y Martes se realizan la mayor cantidad de compras. Al agrupar por franja horaria, se evidencia que la mañana y la tarde son los momentos preferidos por los usuarios para realizar compras en la *web* del supermercado, mientras que los momentos menos preferidos son el martes a la tarde y en la madrugada de lunes a jueves, posiblemente por ser días laborales donde gran parte de los usuarios descansa. Estos hallazgos sirvieron de base para definir las variables temporales y de comportamiento que son de utilidad para las etapas de segmentación de clientes y modelado de recomendaciones. Estos patrones son útiles para la construcción de perfiles de clientes y para que el modelo pueda anticipar el momento del día en que el usuario hará sus compras.

4. Feature Engineering

En este capítulo se detallan las acciones realizadas en cuanto a la preparación de los datos, donde se incluye limpieza, creación de nuevas variables, codificación de variables categóricas, eliminación de valores nulos entre otras modificaciones.

4.1 Transformaciones asociadas al horario y día de la semana

Para facilitar la interpretación posterior de resultados, se agrega la variable `day`, la cual traduce los valores de la variable `order_dow` con el nombre de la semana, de la siguiente forma:

<code>order_dow</code>	<code>day</code>
0	Monday
1	Tuesday
2	Wednesday
3	Thursday
4	Friday
5	Saturday
6	Sunday

Tabla 4.1: Correspondencia entre valores de `order_dow` y la nueva variable `day`.

Luego de la creación de esta nueva variable, se procede a verificar la existencia de valores nulos en el conjunto de datos. Se encontró que los únicos valores nulos existentes en `df` pertenecen a la variable `days_since_prior_order`, la cual indica la cantidad de días transcurridos desde la última compra. Como solamente el ~6.5% de estos registros tenían valores faltantes, se decidió eliminarlos en lugar de imputarlo. De esta forma se evita introducir “ruido” y garantizamos que el sistema de recomendación entrene sobre ejemplos representativos del comportamiento de compra real.

A su vez, se comprueba que no haya registros de compra duplicados en `df`, lo cual es fundamental para evitar sesgos en el análisis.

4.2 Segmentación de Clientes por Cantidad de Órdenes

Sobre la hipótesis existente de que el comportamiento de los clientes varía según la cantidad de órdenes o compras que realizan a lo largo del período de tiempo de análisis, se decidió segmentarlos bajo ese criterio. Para esto, se calculó para cada cliente el valor máximo de pedidos realizados y se anexó esta nueva variable al ds. Primero se agruparon las filas por `user_id` y para cada grupo, se extrajo el valor máximo de `order_number`. Ese valor se almacenó en una nueva variable denominada `max_order`, la cual se agregó al conjunto de datos según el `user_id`.

Como siguiente paso, se mapearon los grupos de usuarios que se desean obtener, marcando conjuntos que se agrupan por máxima cantidad de órdenes realizadas. Dado que la máxima cantidad de órdenes detectada es de 100 órdenes por usuario, se crearon cinco grupos según su número máximo de pedidos, en intervalos de a 20 órdenes.

A continuación, en la tabla 4.2 se muestran los cinco grupos de usuarios definidos según el rango de `max_order`

Grupos	Ordenes Max.
1	1-20
2	21-40
3	41-60
4	61-80
5	81-100

Tabla 4.2: Agrupación de usuarios por rangos según máxima cantidad de órdenes.

4.3 One Hot Encoding

En el contexto del presente trabajo, esta técnica se utiliza para codificar atributos categóricos de los productos y agruparlos según distintos usuarios. La principal motivación para realizar esta técnica, es para poder realizar clusterización mediante *K-means*.

Gran cantidad de modelos de aprendizaje automático pueden trabajar con variables categóricas, siempre y cuando estas se transformen adecuadamente a una representación numérica.[19]

Para convertir las variables categóricas del modelo a variables numéricas, se realizó *One Hot Encoding*. Para esto, se aplicó la función `pd.get_dummies` al conjunto de datos, seleccionando únicamente las variables categóricas `order_number_group`, `department`, `product_name`, `day` y `order_time_list` y eliminando una columna por variable para evitar multicolinealidad, es decir, la alta correlación entre predictores que puede distorsionar el análisis. Como resultado, se obtuvo una tabla en la que cada valor de las variables categóricas mencionadas previamente pasaron a ser nuevas variables *dummy*, es decir que toman valores 0 o 1 según corresponda.

Finalmente, la tabla resultante del *One Hot Encoding* aplicado se unió al conjunto de datos a través de un *merge* sobre `user_id`, de modo que cada usuario conserve sus atributos cualitativos y las nuevas variables *dummy* transformadas. A este nuevo conjunto de datos se le llamó `Supermercado_onehot.csv`, el cual contiene todos los cambios y modificaciones que se han descrito hasta el momento.

Con esta última modificación finaliza la fase de *Feature Engineering* y se continúa con la siguiente aplicación que es la *Clusterización*, es decir, la agrupación de clientes en segmentos homogéneos según sus hábitos de compra. Con el fin de ajustar en una etapa posterior el sistema de recomendación a las particularidades de cada cliente según su perfil.

5. Clusterización

De la etapa de Feature Engineering derivó un conjunto de datos denominado `Supermercado_onehot.csv`, el cual incluye la frecuencia de compra de cada cliente, las preferencias de productos por departamento, entre otros. A partir de este conjunto de datos se inicia la segmentación de clientes. El objetivo de la clusterización es descubrir segmentos de clientes con patrones de compra similares, para luego tomar el resultado de la clusterización y emplearlo como input en el sistema de recomendación. La clusterización es útil previo a realizar un sistema de recomendación, ya que permite agrupar a los clientes según comportamientos de compra similares, lo cual facilita la personalización de las sugerencias.

Para seleccionar la técnica de clusterización más adecuada al comportamiento de compra, se exploraron tres enfoques. Uno de ellos es *K-means*, elegido por su simplicidad y eficiencia en conjuntos de datos grandes y dispersos. Se exploró DBSCAN para detectar densidades y puntos de ruido sin la necesidad de definir en cuantos grupos segmentar los datos y por último se probó HDBSCAN con el objetivo de detectar agrupaciones de densidad variable siguiendo una estructura jerárquica. Tanto DBSCAN como HDBSCAN fueron descartados debido a que generaban o un clúster demasiado grande o demasiados clusters pequeños y fragmentados, con mucha presencia de valores atípicos, lo cual dificulta la interpretación y aplicación práctica.

En el presente capítulo se detallan los procedimientos seguidos para aplicar cada una de las técnicas de clusterización mencionadas y tras su comparación, se define la clusterización final seleccionada.

5.1 Preprocesamiento

Previo a la aplicación del algoritmo de clusterización, se realizó el preprocesamiento del conjunto de datos, de modo de asegurar la relevancia de las variables. Se comenzó seleccionando las variables directamente relacionadas al patrón de compra de los clientes,

dejando de lado variables que no resultan útiles para la segmentación, como los identificadores de cliente o datos demográficos. Luego, se escalaron las variables numéricas a una misma escala, ya que se utilizó *K-means*, que emplea distancias euclídeas en el espacio de características, por lo que las variables que tengan datos muy distintos pueden dominar la formación de clústeres. Se utilizó `StandardScaler` de `scikit-learn`⁴ para realizar la estandarización de las variables a media 0 y desvío estándar 1, asegurando que todas las características contribuyan con peso similar al cálculo de distancias.

Como resultado de este preprocesamiento se obtuvo una matriz de características lista para la clusterización, donde cada fila corresponde a un cliente y los datos se encuentran depurados y normalizados, con la información relevante para el análisis en cuestión.

5.2 Implementacion

Con los datos preprocesados, se procedió a realizar la clusterización empleando el algoritmo *K-means*. Este proceso implicó varias iteraciones. Dado que el número óptimo de clústeres (n) no se conoce de antemano, se utilizó el “método del codo” para determinar un rango inicial, de esta forma se realizó la clusterización para distintos valores de n : 2,3,4 y 5, buscando una separación de grupos sin sobredimensionar la segmentación. Como se detalla en la siguiente sección, para cada valor de n se entrenó un modelo de *K-means*, el cual se ejecutó hasta la convergencia y con un `random_state` definido para garantizar reproducibilidad en las pruebas comparativas.

Para cada clúster se evaluaron tres métricas, el coeficiente de *Silhouette*, el índice de Calinski-Harabasz y el índice de Dunn. *Silhouette* mide qué tan separado está cada punto de otros clústeres en comparación con los de su mismo clúster, indicando que tan bien asignado a un clúster está cada cliente. El índice de Calinski-Harabasz cuantifica la razón entre la dispersión entre clústeres y la dispersión intracluster, indicando si los clústeres están bien separados. El índice de Dunn es un coeficiente que mide la distancia mínima

⁴ Documentación scikit-learn:
<https://scikit-learn.org/stable/>

inter-clúster y la máxima intra-clúster, lo cual permite evaluar si los clústeres están separados entre sí pero son internamente compactos.

Se aplicó PCA para reducir el espacio a 2 dimensiones y capturar la mayor varianza posible, con fines de visualización y análisis, aunque la asignación de clúster se basó en las dimensiones originales. [20]

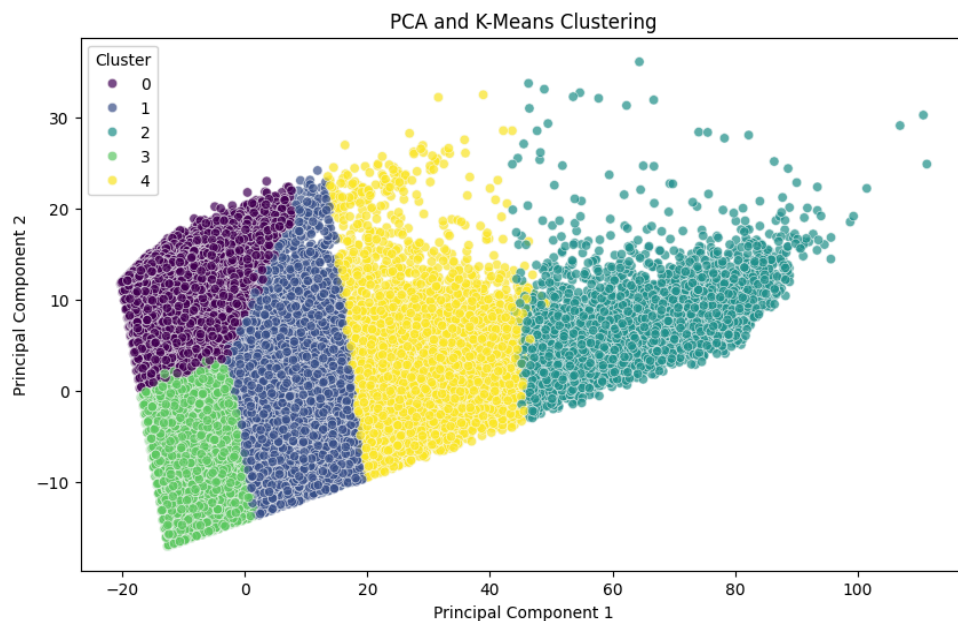


Figura 5.1: Ejemplo de gráfico PCA, correspondiente a *clustering* con $n=5$.

El gráfico anterior corresponde al PCA para $n=5$, donde se puede apreciar que existe una separación razonable entre los grupos.

Además del análisis PCA, se llevaron a cabo visualizaciones univariantes para entender las características de cada clúster. Se graficaron histogramas (Figura 5.2) de variables clave como `max_order` (número máximo de pedidos realizados por el cliente) y `days_since_prior_order` (días transcurridos desde la compra anterior del mismo

cliente) para analizar la distribución de cada atributo dentro de cada grupo. Estos histogramas permitieron caracterizar los clústeres de forma más precisa.

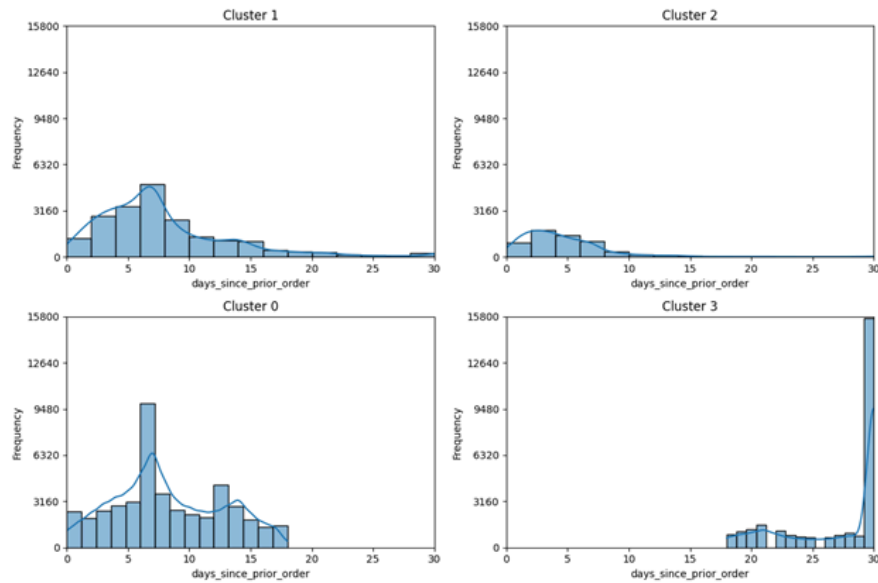


Figura 5.2: Ejemplo de histogramas para la variable `days_since_prior_order` del clustering con $n=4$.

Otro recurso empleado fue la matriz de correlación de Spearman (Figura 5.3), para la cual se calculó la correlación entre la etiqueta del clúster y cada variable categórica. Esto permitió identificar si al aumentar el número de clústeres aumenta o disminuye la probabilidad de consumir productos de determinados departamentos.

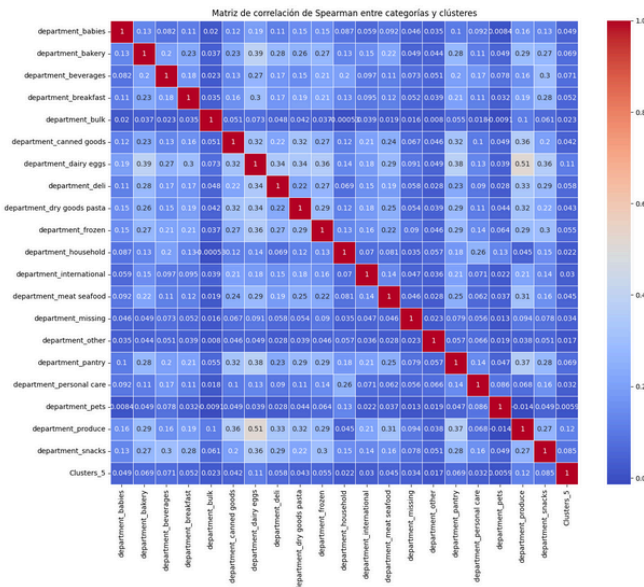


Figura 5.3: Matriz de correlación de Spearman correspondiente a *clustering* con $n=5$.

5.3. Elección del Número de Clústeres

La elección del número óptimo de clúster es fundamental para este análisis. Como se mencionó anteriormente, para ello se emplearon varias herramientas, siendo una de ellas el “método del codo”. Esta es una técnica que evalúa la inercia, es decir la suma de distancias al cuadrado, para distintos valores de n y busca el punto de inflexión donde la mejora comienza a ser menor, indicando el número de clusters adecuado. A partir de ello, se generaron los valores de inercia necesarios para aplicar el método del codo y determinar el valor adecuado de n en *K-means*.

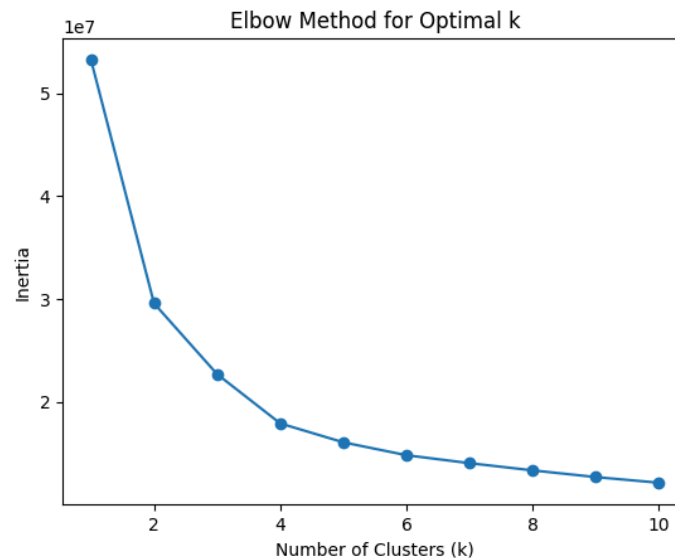


Figura 5.4: Gráfico del método del codo.

Si bien el método del codo orienta al análisis y la búsqueda del número de clústeres óptimo, se emplearon métricas como *Silhouette* y los índices antes mencionados para la evaluación del número óptimo de clústeres. Estas métricas ofrecieron resultados desparejos: *Silhouette* favoreció a una mayor cantidad de clústeres, mientras Dunn sugirió que 3 clústeres implicaba una mejor separación entre grupos y Calinski-Harabasz favoreció a una menor cantidad de clústeres. Por ello, las conclusiones no se guiaron únicamente por un índice sino que se interpretó cada caso por separado.

A continuación, dado que el procedimiento de entrenamiento y evaluación es análogo para todos los valores de n , se presenta un análisis detallado de los casos más representativos

($n=2$, $n=5$ y $n=5$ con reducción de variables), con el objetivo de explorar los fundamentos del análisis que se llevó a cabo para determinar el número óptimo de clústeres para el modelo. El detalle completo está disponible en el repositorio de GitHub del proyecto.

5.3.1 Clusterización $n=2$

En esta sección se analiza la clusterización con dos clústeres, la cual segmenta la base en dos amplios grupos. Para esta clusterización se incluyeron variables relacionadas con el comportamiento de compra de los clientes, junto con las categorías de productos resultantes del *one hot encoding*. Luego se aplicó *StandardScaler* para evitar que ciertas variables dominen el cálculo de distancias. Una vez obtenidos los datos escalados, se configuró *K-means* con `n_clusters=2` y `random_state=100` para asegurar reproducibilidad.

Se realizó un análisis gráfico mediante un diagrama de dispersión (Figura 5.5), con el objetivo de obtener una idea preliminar de la separación de clústeres:

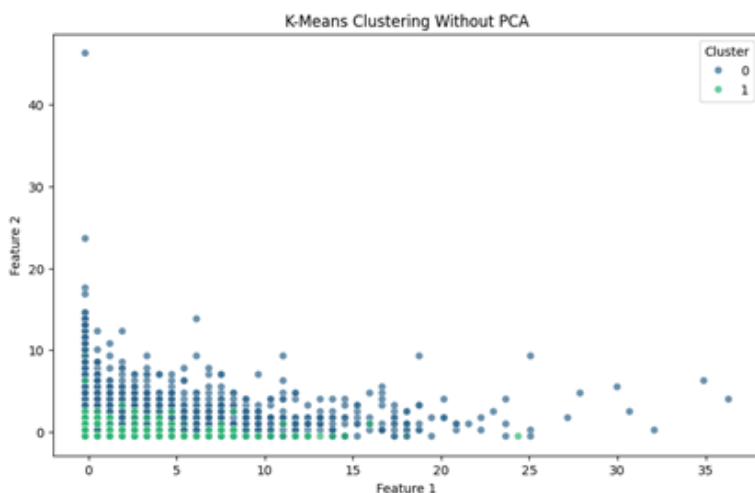


Figura 5.5: Gráfico de clúster con $n=2$ sin reducción de dimensionalidad.

Del análisis gráfico surge que el clúster 0 tiende a abarcar valores moderados y altos mientras que el clúster 1 abarca valores más bajos. Sin embargo, se aprecia un solapamiento considerable sin una frontera clara entre clústeres.

Luego, mediante un gráfico de barras apiladas (Figura 5.6), se analizó la proporción de clientes en cada clúster por departamento. Del gráfico se aprecia que el clúster 0 concentra la mayor parte de las compras en casi todos los departamentos, lo cual indica que tiene dominancia respecto al clúster 1.

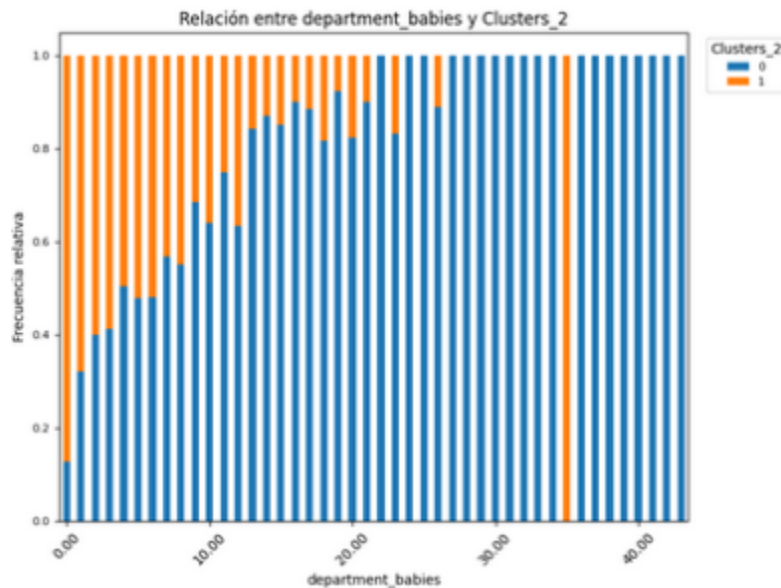


Figura 5.6: Ejemplo de gráfico de barras apiladas por departamentos para la variable department_babies.

Posteriormente, se realizó la visualización de la matriz de correlación, la cual sugiere que la clasificación en dos clústeres no presenta una fuerte asociación con las preferencias de por categorías de productos, lo cual indica que agrupar en dos segmentos no logra capturar adecuadamente los patrones de compra.

Por último se graficaron histogramas para la variable days_since_prior_order, graficando la distribución para cada clúster (Figura 5.7). Ambos clústeres se distribuyen a lo largo de 30 días, con el clúster 0 mostrando un pico en los 7 días mientras que el clúster 1 presenta dos picos, uno a los 7 días y otro a los 30, aunque más distribuido a lo largo del eje. Esto implica que ambos tienen comportamientos de compra similares, con diferencia en el tamaño del clúster .

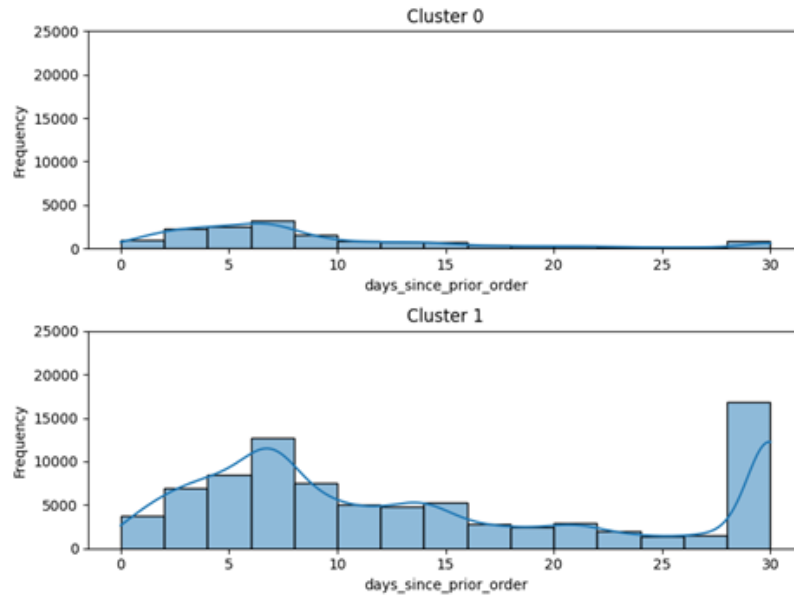


Figura 5.7: Histogramas para la variable `days_since_prior_order` del *clustering* con $n=2$.

Si bien la clusterización con $n=2$ permitió un primer acercamiento y logró separar la base en un segmento amplio y otro más pequeño, no se obtuvo una diferenciación adecuada de los perfiles de clientes. Del análisis anterior se concluye que existe un solapamiento de comportamientos, por lo cual la segmentación con $n=2$ es insuficiente para el objetivo del análisis.

5.3.2 Clusterización $n=5$

Con el objetivo de mejorar la segmentación de clientes, se continuó con el análisis con una segmentación con $n=5$. Se buscó comprobar si un clúster adicional permitía captar con mayor precisión las diferencias de comportamiento de compra, reduciendo el solapamiento que se observó en las segmentaciones previas ($n=3$ y $n=4$).

Para este análisis, se emplearon las mismas variables y procedimientos que en las iteraciones anteriores. Se comenzó con una visualización mediante PCA, donde se aprecian cinco clústeres, algunos con fronteras más definidas que otros. Los clústeres 2 y 4 presentan mayor grado de solapamiento entre sí y a su vez mayor variabilidad interna. Mientras que los clústeres 0, 1 y 3 muestran límites más definidos y compactos.



Figura 5.8: Gráfico de clustering con $n=5$ y reducción de dimensionalidad.

A continuación, se analizó la participación de cada clúster por departamento. Se observa que todos los clústeres tienen participación en todos los departamentos, pero se identifican algunas características diferenciales. En el departamento produce, se observa una alta participación del clúster 2 respecto a la de los demás clústeres, lo cual indica que el clúster 2 tiene preferencia por productos frescos y naturales.

La categoría snacks también muestra variaciones marcadas entre los clústeres, con algunos con mucha participación, como en los clústeres 1 y 3, implicando que estos tienen hábitos de compras impulsivas o de productos que no son frescos. En contraste, los departamentos como Household y Bakery tienen una participación más homogénea, indicando que estos no resultan útiles para distinguir entre segmentos.

Por otro lado, el clúster 0 tiende a concentrar sus compras en ciertos departamentos, lo que podría indicar una canasta muy definida de productos. Los clientes del clúster 1 muestran una preferencia combinada hacia departamentos como Produce, Snacks, Dairy Eggs, entre otros, lo que podría interpretarse como un perfil enfocado en artículos frescos. El clúster 2 no presenta un patrón de preferencias claro, lo que sugiere una conducta de compra más variada. El clúster 3 denota preferencia por los departamentos Dairy Eggs, Bakery y similares, lo que indica que son clientes fieles y con preferencias claras y

consistentes. Por último, el clúster 4 domina la mayor parte de los departamentos respecto a los demás clústeres, indicando una conducta de compra diversificada.

Luego se realizaron los histogramas por clúster de la variable `days_since_prior_order`, donde se observa que los clústeres muestran diferencias de comportamiento entre sí respecto a la frecuencia de pedidos en función de cuántos días pasan entre cada orden previa

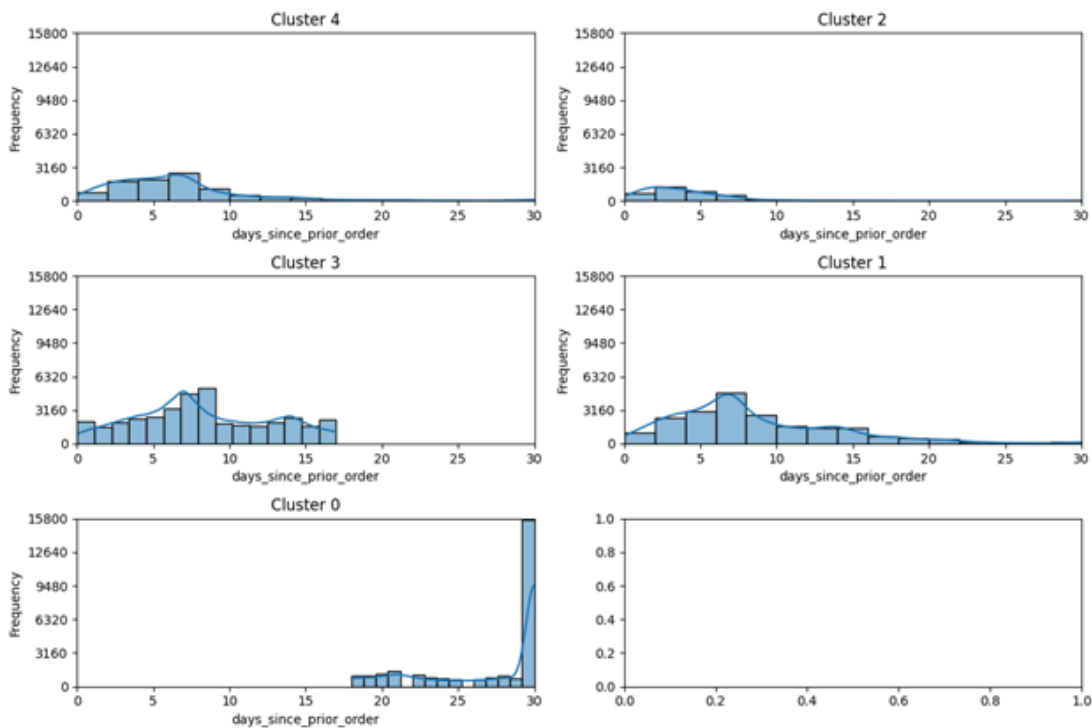


Figura 5.9: Histogramas para la variable `days_since_prior_order` del *clustering* con $n=5$.

Para el clúster 0 se observa que los clientes de ese segmento realizan principalmente compras mensuales. El clúster 1 tiene una frecuencia mediana de compra, con un rango más amplio que va de 0 a 15 días y luego poca frecuencia de pedidos hasta los 30 días. El pico de frecuencia se encuentra en los 7 días por lo que indica compras mayormente semanales. El clúster 2 tiene una concentración de compras entre los 0 y 7 días, con un pico en 3 días, lo cual indica que son clientes que realizan compras rápidas y con mucha frecuencia. El clúster 3 tiene un pico en los 8 días con una distribución que se extiende poco más de los 15 días, aunque con menor frecuencia. Por lo que indica compras semanales o hasta quincenales. El clúster 4 presenta una concentración de compras entre

los 2 y 7 días, representando una compra frecuente pero no tan inmediata como la del clúster 2.

Luego se realizaron los histogramas para visualizar la distribución de `max_order`. El clúster 0 muestra una alta distribución para valores bajos de `max_order` y decrece gradualmente hasta valores de `max_order` de 30, por lo que se compone de pocos usuarios con muchos pedidos y muchos usuarios con hasta 30 pedidos. El clúster 1 presenta menor cantidad de usuarios con pocos pedidos, la mayor cantidad de pedidos se concentra entre los 15 y 30 pedidos. El clúster 2 es el más reducido, con usuarios nuevos con pocas órdenes acumuladas. El clúster 3 tiene una proporción elevada de usuarios con valores bajos de órdenes. Por último, el clúster 4 refleja un pico de `max_order` en los 40 pedidos, reflejando clientes con un elevado historial de compras.

En síntesis, la segmentación con $n=5$ logró una mejor diferenciación entre perfiles de clientes. Las diferencias en frecuencia de compra, historial de pedidos y categorías de productos reducen el solapamiento observado en versiones anteriores y permiten una interpretación más amplia de los patrones de consumo.

5.3.3 Clusterización $n=5$ con Input Reduction

En esta sección se presenta un análisis final de la segmentación, utilizando cinco clústeres pero con una reducción del conjunto de variables. El objetivo de este apartado es evaluar si una sección más acotada de atributos mejora la calidad del modelo. A continuación, se detallan los resultados obtenidos.

Se comenzó con el análisis PCA, donde se observa que los clústeres se agrupan de manera diferenciada, con una estructura bien definida y densidad interna coherente. El clúster 1 y el clúster 4 son los que tienen mayor dispersión y se extienden un poco más en el plano. El clúster 0 y el 3 cuentan con una distribución más compacta y bien delimitada. El clúster 2, por su parte, muestra una dispersión intermedia en comparación al clúster 0 y 3. Esto indica que la segmentación es consistente con grupos visiblemente separados.

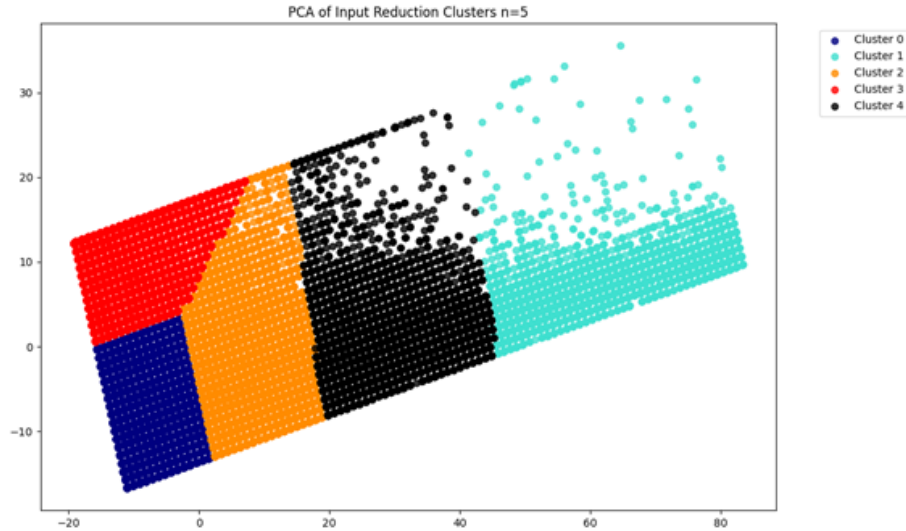


Figura 5.10: Gráfico de *clustering* con $n=5$ con reducción de variables y dimensionalidad.

A continuación, se realizó un gráfico de boxplots para representar las distancias mínimas de cada punto al centroide de su clúster. En este gráfico se observa que los clústeres 0 y 3 tienen una mediana relativamente baja, donde el clúster 0 presenta poca dispersión, indicando una segmentación compacta. El clúster 1 tiene cuartiles más separados, indicando variabilidad interna en las distancias. El clúster 2 exhibe una dispersión moderada, mientras que el clúster 4 es el que presenta mayor mediana y amplia dispersión, con presencia de outliers y variabilidad interna. En conclusión, la mayoría de los clústeres presenta una mediana relativamente baja, lo que indica una asignación adecuada de puntos a sus centroides. Las conclusiones generadas coinciden con lo observado en el gráfico de PCA.

Luego se realizó un gráfico de barras apiladas, donde se visualiza la participación de cada clúster por departamento, indicando su distribución. Este análisis permitió obtener conclusiones globales útiles para interpretar y definir cada clúster. Todos los clústeres participan en todos los departamentos, lo cual indica que las preferencias no están sesgadas hacia un clúster específico. El clúster 0 y el clúster 1 tienen una proporción considerable en todos los departamentos, lo que sugiere que realizan compras más generales sin enfocarse en un departamento específico. El clúster 4 sin embargo, muestra un comportamiento de compra más especializado, con preferencias más marcadas en ciertas categorías.

Siguiendo con el análisis, se realizaron los histogramas para la variable `days_since_prior_order` desglosado por clústeres. El clúster 0 se concentra entre los 5 y 15 días, indicando que son clientes que realizan compras con una cierta regularidad. El clúster 1 tiene una frecuencia más elevada, se concentra en intervalos de entre 0 y 10 días, con muy pocos casos en intervalos más largos de hasta 25 días. El clúster 2 tiene un pico en los 7 días, pero combinado con algunos lapsos más largos, indicando un comportamiento híbrido con preferencia de compras semanales. El clúster 3 agrupa compradores principalmente mensuales y bastante compacto. Por último, el clúster 4 muestra una distribución con picos en intervalos cortos, pero extendiéndose a intervalos de compra más amplios, componiéndose de clientes con comportamientos variados o con hábitos de compra más variables.

Luego se graficaron los histogramas para la variable `order_hour_of_day`. Mediante esta representación se observa que todos los clientes realizan sus compras entre las 6am y las 22pm, aunque con algunas diferencias según el segmento al cual pertenecen. El clúster 0 representa un comportamiento más unificado enfocando sus compras en franjas matutinas o en mitad de la jornada. El clúster 1 muestra un pico de compras entre las 8 y las 12 hs, mientras que el clúster 2 presenta dispersión con picos en la mañana y en la tarde. El clúster 3, se comporta de forma similar al 0 pero con una distribución más concentrada, y el clúster 4 abarca una franja horaria amplia, indicando la presencia de consumidores con distintos hábitos de compra.

De forma de continuar con el análisis se realizó el gráfico de histogramas para la variable `max_order`.

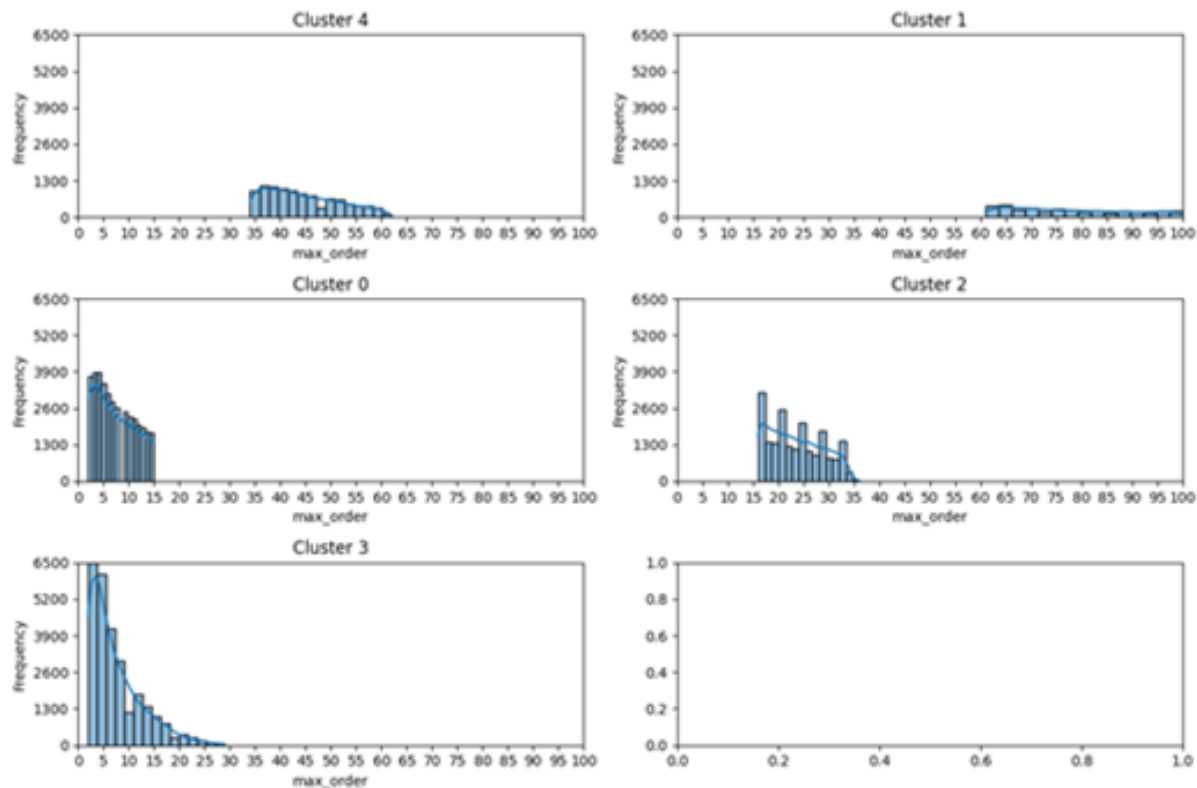


Figura 5.11: Histogramas para la variable `max_order` del clustering con $n=5$.

El *clúster 0* representa a clientes nuevos o poco frecuentes, ya que su distribución se limita a rangos bajos. El *cluster 1* es el que presenta el conjunto de clientes con mayor cantidad de compras máximas realizadas. Por otro lado, el *clúster 2* presenta un rango medio heterogéneo, con picos de entre 15 y 20 órdenes máximas, y el *clúster 3* presenta valores intermedios sin extenderse a mucha cantidad de órdenes. Finalmente, para el *clúster 4* las órdenes se concentran en niveles de entre 35 y 55 órdenes máximas.

De este análisis se concluye que la clusterización con $n=5$ y reducción de variables ofrece una segmentación robusta logrando capturar correctamente los comportamientos de los clientes que componen cada segmentación. A partir de esto, se encuentran 5 grupos de clientes según el clúster al que pertenecen:

- **El clúster 0:** se compone de compradores nuevos o poco frecuentes. Son consumidores que tienen un bajo historial de compras, con un máximo de 15 órdenes acumuladas. Tienden a realizar compras pequeñas y específicas,

posiblemente enfocadas en productos básicos o de prueba.

- **El clúster 1:** representa a compradores leales, posiblemente se trata de familias con alto ritmo de consumo y predominancia en departamentos como Breakfast y Babies.
- **El clúster 2:** sugiere que se compone por compradores ocasionales que realizan compras diversificadas. Este grupo puede representar a estudiantes y adultos jóvenes.
- **El clúster 3:** engloba consumidores que realizan compras mensuales y planificadas.
- **El clúster 4:** representa consumidores recurrentes y especializados, que realizan compras de gran volumen, posiblemente para fines comerciales o de reposición.

Luego de explorar diferentes segmentaciones (con $n=2$ hasta $n=5$), se concluyen los siguientes resultados:

- **Para $n=2$** se obtuvieron dos segmentos muy generales, que si bien sirvieron para capturar la distinción entre clientes nuevos y los ya establecidos resultó muy general como para poder distinguir características específicas de compra.
- **Para $n=3$** la segmentación incluyó a clientes inactivos en un clúster, a los clientes leales en otro y a los de frecuencia intermedia en un tercero. Sin embargo, la diversidad de comportamiento dentro de cada grupo resultaba elevada.
- **Para $n=4$** se identificaron cuatro grupos de clientes con comportamientos diversos, aunque con cierto solapamiento entre clústeres, lo cual dio indicio a que aún era necesaria una mayor diferenciación.

- **Para $n=5$** se logró una mejor segmentación de clientes, logrando resolver el solapamiento que se detectó en $n=4$.
- **Para $n=5$ con reducción de variables** se obtiene la segmentación más sólida para el análisis. Ya que permite identificar perfiles de clientes bien diferenciados, con hábitos de compra claros y comportamientos distinguibles en cuanto a frecuencia, historial de pedidos y categoría de productos preferida. Además, al reducir la cantidad de variables a las más representativas se mejora la interpretabilidad del modelo sin sacrificar la capacidad de segmentación.

5.4 Clusterización Resultante

Luego de definir $n=5$ se procedió a entrenar el modelo final de clusterización y a refinar ligeramente la selección de variables hacia las principales, dejando de lado algunas variables categóricas. Esto se debe a que se observó que estas últimas podían introducir ruido y se consideró conveniente incorporarlas en análisis posteriores. Las variables seleccionadas para el modelo final fueron: `max_order`, `order_hour_of_day` (hora del día en la que se realizó la compra.), `days_since_prior_order` y `reordered` (indica si un producto se volvió a comprar o no). Luego de esto, se normalizaron las variables y se volvió a ejecutar *K-means* con $n=5$ sobre este subconjunto de variables. Tras obtener las etiquetas y los centroides, se validó la calidad del modelo con las métricas mencionadas anteriormente.

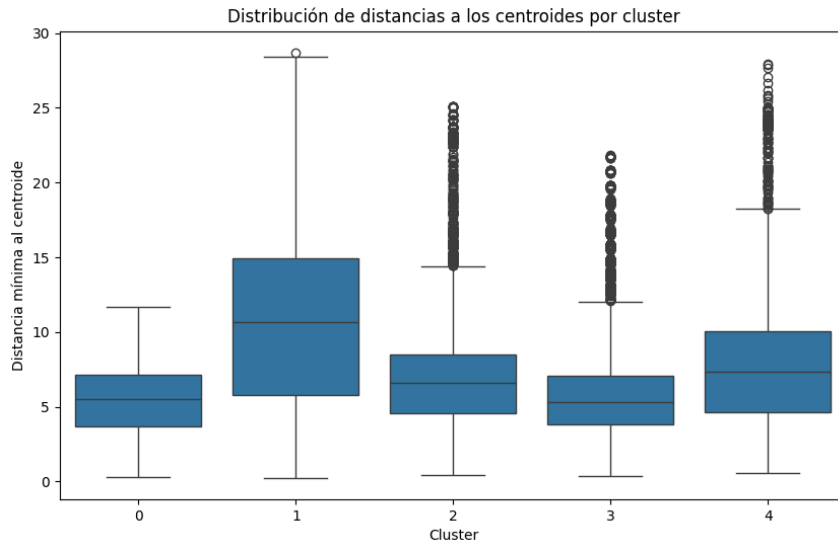


Figura 5.12: Diagrama de caja de la distribución de distancias a los centroides por clúster para $n=5$.

Del análisis gráfico de la distancias a los centroides por clúster, se observa que la mayor parte de los clústeres muestra un nivel de dispersión moderado y que la separación global del modelo *K-means* con $n=5$ resulta efectiva para segmentar el perfil de los clientes.

En cuanto a las métricas, los resultados fueron los siguientes:

- **Silhouette** arrojó un valor cercano a 0.5 lo cual indica que existe una separación moderada entre clústeres.
- **Calinski-Harabasz** dio como resultado un índice elevado (177632.3243), lo cual indica que los clústeres son densos internamente y razonablemente separados entre sí.
- **Dunn** dio un valor bajo (0.0048) lo cual indica que algunos clústeres podrían estar relativamente próximos.

Si bien esta última métrica sugiere una posibilidad de mejora, los demás indicadores, sumado a la interpretación cualitativa, respaldan la decisión de que la clusterización con $n=5$ cumple con el objetivo principal de segmentar a los clientes según sus patrones de compra.

Por lo tanto, para $n=5$ se identificaron los siguientes grupos de clientes:

- **Clúster 0:** "Compradores Nuevos o Infrecuentes"
- **Clúster 1:** "Compradores Leales"
- **Clúster 2:** "Comprador Ocasional Que Realiza Compras Diversificadas"
- **Clúster 3:** "Compradores Mensuales"
- **Clúster 4:** "Consumidor Recurrente y Especializado"

Sumado a la metodología implementada con *K-means*, se probó una clusterización con DBSCAN, aunque no fue utilizada como parte del modelo final debido a resultados insatisfactorios. El algoritmo DBSCAN se diferencia de *K-means* en el sentido de que no requiere especificación previa de un número de clústeres, sino que identifica agrupaciones en función de la densidad de puntos en el espacio de características, asignando a cada observación un DBSCAN en función de dos parámetros principales. El motivo por el cual se decidió testear con DBSCAN es porque resulta útil para datos complejos y de gran volumen.

Debido a la magnitud y diversidad del conjunto de datos de compras de supermercado con el cual se trabajó, previo a aplicar DBSCAN se tomó una muestra estratificada. Además, se normalizaron los datos para evitar que la escala de alguna variable denominara el cálculo de la densidad. Luego se probaron múltiples valores para las variables `eps` y `min_samples` y diferentes conjuntos de datos, donde `eps` se utiliza para determinar la distancia máxima para la cual dos puntos se consideran cercanos y `min_samples` representa el número mínimo de puntos requeridos para formar un clúster alrededor de un núcleo. También se aplicó PCA para reducir la dimensionalidad y visualizar los resultados en dos dimensiones, y se exploraron variantes como el HDBSCAN, que expande el enfoque de densidad.

A modo de ilustrar la asignación de etiquetas se generaron distintos gráficos, tanto en el espacio original como en el reducido a dos componentes principales. En estos se observó que DBSCAN formó un clúster principal que abarcaba la mayor parte de las observaciones, mientras que varios puntos fueron considerados como “ruido”, por lo que no formaban parte de ningún clúster. A su vez, se generaron varios clústeres pequeños compuestos por pocos puntos difíciles de relacionar con patrones de compra específicos.

A pesar de que HDBSCAN cuenta con la ventaja de que no requiere un parámetro de densidad fijo y que detecta agrupaciones de distinta densidad de forma jerárquica, en la práctica generó un gran número de clusters muy pequeños y fragmentados, con gran cantidad de valores atípicos sin un patrón claro. Este exceso de partición y la gran heterogeneidad dentro de cada uno de ellos, dificultó la interpretación y el uso práctico de los resultados.

Dados los resultados de estas clusterizaciones, se optó por no utilizar DBSCAN ni su variante. *K-means*, en cambio, ofreció una segmentación más homogénea y útil para interpretar el comportamiento de los clientes.

6. Métricas

Tras la segmentación de clientes analizada en el capítulo anterior, se identificaron cinco perfiles diferenciados según hábitos y comportamientos de compra. Esta clusterización previa agrupa a los usuarios en segmentos homogéneos, lo cual resulta esencial para personalizar cada modelo de recomendación y mejorar su eficacia.

Al entrenar los modelos con las características específicas de cada grupo, no solo se comprende mejor la diversidad de consumidores, sino que también se sientan las bases para un sistema predictivo realmente adaptado a distintos tipos de usuario, permitiendo un nivel mayor de customización en las recomendaciones.

En este capítulo, se definirán las métricas que se utilizarán para evaluar los modelos de recomendación, y a partir del análisis de estas métricas, se determinará cuál es la más adecuada para la comparación de los modelos.

Las métricas utilizadas para evaluar los modelos de recomendación son *RMSE* (*Root Mean Squared Error*), *Precision*, *Recall*, y *F1-score*, dado que se trata de un problema de clasificación.

El *RMSE* (*Root Mean Squared Error*) mide la diferencia promedio entre los valores predichos por un modelo y los valores reales, penalizando más los errores grandes. Se calcula como la raíz cuadrada del promedio de los errores al cuadrado entre las predicciones y los valores verdaderos.

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (6.1)$$

Siendo y_i los valores reales, $\hat{y}_i$ los valores predichos por el modelo, y n el número total de predicciones.

Esta métrica, permite medir la calidad de las predicciones, pero lo hace desde una perspectiva numérica, ya que evalúa qué tan cerca están los valores predichos de los reales. Sin embargo, no permite evaluar qué tan relevante es la recomendación para el usuario. Esto nos lleva a las métricas *Precision*, *Recall*, y *F1-score* que evalúan efectivamente la calidad de las recomendaciones, pero de manera distinta.

Por un lado, *Precision* mide la proporción de verdaderos positivos sobre el total de instancias realizadas por el modelo. Es decir, de todas las recomendaciones que el modelo consideró relevantes, cuántas realmente lo eran.

$$\text{Precision} = \frac{\text{Verdaderos Positivos}}{\text{Verdaderos Positivos} + \text{Falsos Positivos}} \quad (6.2)$$

Por otro lado, *Recall* mide la proporción de verdaderos positivos que fueron efectivamente recomendados. Es decir, mide la capacidad del modelo de recuperar productos relevantes de todos los elementos que deberían haber sido recomendados.

$$\text{Recall} = \frac{\text{Verdaderos Positivos}}{\text{Verdaderos Positivos} + \text{Falsos Negativos}} \quad (6.3)$$

Dada la gran cantidad de departamentos y productos en el *dataset*, el número de productos relevantes para el usuario es menor que los no relevantes, lo cual provoca un desbalance de clases. En este escenario, métricas como *Precision* y *Recall* no son las más acertadas para evaluar el modelo por sí solas, ya que pueden llevar a conclusiones erróneas.

A modo de ejemplo, un modelo que recomienda siempre los tres productos más comprados (produce, dairy eggs, snacks) va a tener precisión alta ya que la gran mayoría de los usuarios los compran. Por otro lado, si el usuario también es un consumidor de otras categorías no tan usuales (pets, household, babies), el modelo nunca las recomendará, dejando sin cubrir categorías importantes para el usuario.

En el caso del recall, si tenemos un modelo que le recomienda al usuario una gran cantidad de productos (100), intentando de no dejar afuera ninguna categoría dentro del historial de compra del consumidor. El modelo tendrá un recall alto, ya que encuentra los

productos relevantes, pero al mismo tiempo le recomienda al usuario una gran número de productos que quizás fueron compras esporádicas

Dado que estas métricas son insuficientes cuando se utilizan de forma aislada (especialmente en escenarios con clases desbalanceadas), es necesario recurrir a la métrica *F1-score*. Esta métrica se calcula como la media armónica entre la *Precision* y el *recall*, contemplando ambas de forma simultánea, penalizando modelos que presentan gran desequilibrio entre ellas. Esto permite capturar el *trade-off* entre el *Recall* y *Precision*.

$$F1Score = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \quad (6.4)$$

Por lo tanto, se utilizará el *F1-score* como métrica principal para la comparación de los modelos de recomendación desarrollados en este trabajo.

En los próximos capítulos, se indaga sobre las implementaciones de los distintos modelos, y la comparación de estos mismos según las métricas definidas, y la elección del mejor modelo según la métrica principal

7. Implementación de Algoritmos / Sistemas de Recomendación

Una vez definidas las métricas de evaluación y seleccionado el *F1-score* como la métrica principal para la comparación de los modelos, se da paso a la implementación de los mismos. En este capítulo, se trabaja sobre un conjunto de datos enriquecido con la segmentación previa, con el objetivo de entrenar y evaluar modelos capaces de generar recomendaciones personalizadas para cada tipo de usuario.

Para poder avanzar hacia el objetivo principal del proyecto, la implementación del sistema de recomendación, se generó un conjunto de datos derivado de la clusterización con $n=5$ y reducción de variables. El mismo incorpora las etiquetas del clúster al que pertenece cada cliente, sirviendo como base para entrenar distintos modelos de recomendación. Para esto, se asignaron etiquetas descriptivas de cada clúster, las cuales fueron incorporadas como un nuevo atributo. Además, se eliminaron de este nuevo conjunto de datos las variables que mostraron escaso poder predictivo sobre la variable objetivo. Este nuevo conjunto de datos se llamó `Cluster5_1_items.csv`, de ahora en más “conjunto de datos para recomendaciones”.

En este capítulo se describe con detalle la implementación y evaluación de diversos algoritmos de recomendación, apoyado por ChatGPT para la generación de código. Entre estos algoritmos se encuentran NMF, TFRS y Surprise, cada uno con características particulares que serán analizadas en profundidad para determinar su aplicabilidad y eficacia en la generación de recomendaciones. Asimismo, se probaron otros enfoques como Implicit y LightFM, cuya ejecución no pudo completarse por motivos que se detallan al final de esta sección.

7.1 Balanceo del dataset

Antes de avanzar con el entrenamiento de los modelos de recomendación, fue necesario realizar un preprocesamiento adicional sobre el conjunto de datos para recomendaciones. Durante el análisis exploratorio se observó un desbalance significativo en la variable

objetivo department, que representa los distintos departamentos o categorías de productos adquiridos por los usuarios. Esta desproporción entre clases puede generar sesgos en los modelos de aprendizaje supervisado, afectando negativamente su capacidad para aprender patrones representativos de las clases minoritarias.

Para abordar esta problemática se aplicó la técnica de sobremuestreo SMOTE (abordada en la sección 2.4.1 del presente trabajo).

En primer lugar, se realizó una codificación de todas las variables categóricas, incluyendo la variable objetivo, para permitir su procesamiento numérico. Esta codificación fue necesaria ya que la herramienta SMOTE requiere que los datos estén representados en formato numérico.

Luego, se separó la variable objetivo, que en este caso fue la variable department. A partir de esa segmentación, se aplicó SMOTE con una semilla aleatoria fija para asegurar que los resultados fueran consistentes en cada ejecución. Esta técnica generó nuevas observaciones sintéticas, utilizando las características existentes en las clases menos representadas. De este modo, se alcanzó una distribución balanceada en la variable objetivo, asegurando que cada departamento estuviera representado con una cantidad similar de registros.

Finalmente, se reconstruyó un nuevo dataset denominado `Cluster5_1_balanced`, integrando los datos re-muestreados y volviendo a transformar las etiquetas a su forma original para facilitar su interpretación. Por último, se analizó la nueva distribución para confirmar la efectividad del procedimiento, y el conjunto de datos balanceado fue exportado para su uso en los modelos de recomendación que serán analizados en las secciones siguientes.

7.2 Modelo NMF

El algoritmo NMF constituye una técnica de descomposición matricial utilizada en sistemas de recomendación basados en filtrado colaborativo, tal como se detalló en la sección 2.5.1 del presente trabajo. En este capítulo se describe su aplicación práctica sobre el subconjunto de datos correspondiente al clúster 5, en sus dos versiones, tal como se mencionó a principios de este capítulo, por un lado, la versión original (Cluster5_1_items), sin aplicar técnicas de balanceo; y por otro, una versión balanceada. A partir de estas dos variantes, se construyeron dos modelos independientes, lo que permitió analizar el impacto del balanceo de clases en el desempeño del algoritmo y su capacidad para generar recomendaciones relevantes.

Preparación de los datos:

Para comenzar el armado de los modelos, se generó una matriz usuario-producto, cuyas columnas representan distintos product_id y cada fila un user_id distinto. Los valores de la matriz representan si el producto fue comprado nuevamente con “1” y si no, “0”. La figura 7.1 ilustra un ejemplo simplificado de cómo es la estructura de la matriz.

		product_id			
user_id		product_1	product_2	product_3	product_4
	user_1	1	0	0	1
	user_2	0	1	1	0
	user_3	1	1	0	0
	user_4	0	1	1	0
	user_5	1	0	0	1

Figura 7.1: Ejemplo matriz usuario-producto para modelo NMF. [3]

La matriz creada es de dimensión 37973×134 (37.973 usuarios por 134 productos). Se detecta que aproximadamente el 96% de los valores de la matriz son representados con

“0”, esto se debe a que la mayoría de los productos no fueron pedidos de nuevo por el mismo usuario. Esto se debe a que al tener una gran variedad de productos, es razonable que no todos los productos sean pedidos nuevamente por cada usuario en el período comprendido por el *dataset*. Debido a esto, se decide utilizar una matriz dispersa por fila, lo cual ayudará a trabajar de forma más eficiente, optimizando el almacenamiento y procesamiento, almacenando solo las posiciones y los valores no nulos [21].

Entrenamiento del modelo:

Para ambas versiones del dataset, se decidió llevar a cabo el mismo procedimiento en el modelado. En primer lugar, se divide el conjunto de datos en entrenamiento (80%) y prueba (20%) para el entrenamiento del modelo con los siguientes parámetros iniciales:

- **Componentes:** 50
- **Inicialización:** random
- **Algoritmo de Optimización:** cd
- **Función de Pérdida:** frobenius
- **Max. Iteraciones:** 200

Optimización del modelo:

Con el objetivo de mejorar la calidad de las recomendaciones del modelo, se implementó el método *grid search* para iterar múltiples combinaciones de hiperparámetros del modelo, incluyendo el número de componentes latentes, el tipo de inicialización, el algoritmo de optimización (*solver*), la función de pérdida y la cantidad máxima de iteraciones.

Para cada configuración, se entrenó un modelo y se evaluaron las siguientes métricas: RMSE , *Precision*, *Recall*, *F1-score*, y el tiempo de entrenamiento. Esta evaluación permite identificar la mejor combinación de hiperparámetros, registrando cada iteración en una tabla, para su posterior análisis comparativo.

Al correr el *grid search*, y ordenando por *F1-score*, se obtiene que la mejor combinación de hiperparámetros para los modelos con el conjunto de datos balanceado y sin balancear son los siguientes:

- **Componentes:** 100
- **Inicialización:** nndsvda
- **Algoritmo de Optimización:** mu
- **Función de Pérdida:** kullback-leibler
- **Max. Iteraciones:** 300

Resultados obtenidos con cada modelo:

Una vez definidos los hiperparámetros, se corren ambos modelos (*baseline* y optimizado) utilizando el dataset sin balancear y el balanceado. Para evaluar los modelos, se utilizaran las métricas establecidas en el Capítulo 6.

Conjunto de datos sin balancear:

En cuanto a los resultados del modelo entrenado con el dataset sin balancear, se observa que su versión optimizada presenta mejoras en todas las métricas evaluadas:

Métrica	Baseline	Optimizado	Mejora (%)
RMSE	0.0652	0.0291	-55.37%
Precision	0.6669	0.7055	5.79%
Recall	0.7838	0.8551	9.10%
F1-score	0.7207	0.7731	7.27%

Tabla 7.1 Resultados de las métricas para el modelo base y optimizado con datos no balanceados.

Conjunto de datos balanceado:

Al igual que en el caso anterior, se observan mejoras en todas las métricas al comparar el modelo optimizado con el modelo base:

Métrica	Baseline	Optimizado	Mejora (%)
RMSE	0.082	0.0457	-44.27%
Precision	0.4062	0.5197	27.94%
Recall	0.6839	0.928	35.69%
F1-score	0.5096	0.6663	30.75%

Tabla 7.2 Resultados de las métricas para el modelo base y optimizado con datos balanceados.

Resultados:

Los resultados obtenidos luego de ejecutar los modelos con el conjunto de datos sin balancear, muestran variaciones significativas sobre las métricas arrojadas por los modelos ejecutados con el conjunto de datos balanceados.

Métrica	Sin Balancear	Balanceado	Mejora (%)
RMSE	0.0291	0.0457	-36.32%
Precision	0.7055	0.5197	35.75%
Recall	0.8551	0.928	-7.86%
F1-score	0.7731	0.6663	16.03%

Tabla 7.3 Comparativa de los mejores resultados de cada modelo.

El modelo entrenado con el dataset sin balancear logra mejores resultados en la mayoría de las métricas. A pesar de que el balance de clases suele ser beneficioso en contextos supervisados, en el caso de NMF, que opera sobre patrones latentes de interacciones, los datos no balanceados pueden preservar mejor la estructura natural del comportamiento de los usuarios. Esto puede traducirse en una mayor precisión y, en consecuencia, en un mejor *F1-score*, como se evidencia en los resultados obtenidos.

7.3 Tensor Flow Recommender

En esta sección se detalla la implementación realizada del sistema de recomendación con TFRS. Para la construcción de este sistema, se partió de un modelo base y se realizaron ajustes hasta llegar a una versión optimizada y con datos balanceados.

En primera instancia, se elaboró un modelo base, limitado al conjunto de datos para recomendaciones (el cual representa un grupo homogéneo de usuarios), acotando el conjunto de datos original. El modelo base implementa una arquitectura *two-tower*. Donde cada subred aprende *embeddings* de usuarios y productos, capturando sus preferencias en espacios latentes. Las capas de *embedding* toman dimensiones moderadas, y durante la inferencia se calcula un score como el producto interno de los vectores latentes. Para evaluar la calidad de las predicciones, el conjunto de datos se divide en dos partes, una de entrenamiento y otra de validación, donde se evalúan métricas como *RMSE*, *Precision*, *Recall*.

A continuación, se procedió a optimizar los parámetros con el fin de mejorar el rendimiento del modelo. Para ello, se utilizó Keras Tuner⁵, herramienta que prueba de forma automática múltiples configuraciones. Entre los hiperparámetros ajustados por el *tuner* se encuentran la dimensión de los *embeddings*, el número de capas densas, la función de activación, la tasa de aprendizaje y los parámetros de regularización. Como resultado, el *tuner* identificó una combinación óptima de hiperparámetros. Al re-entrenar con esos valores, el modelo con hiperparámetros optimizados demostró mejoras en la evaluación a través de las métricas implementadas, se redujo el error de predicción (*RMSE*) y aumentó la capacidad de aciertos en recomendaciones relevantes (*Precision*, *Recall* y *F1-score*).

Tras optimizar la arquitectura inicial y los hiperparámetros con Keras Tuner, se reentrenó el mismo modelo sobre la versión balanceada del conjunto de datos. De este modo se obtuvo el efecto del desbalance de clases en las métricas analizadas.

⁵ Documentación Keras Tuner:
https://keras.io/keras_tuner/

7.3.1 Detalles Técnicos del Modelo

Para iniciar con el primer modelo de recomendación con TFRS se realizó un preprocesamiento que consistió en convertir el *DataFrame* de Pandas a un objeto con la estructura central que ofrece TensorFlow para representar y procesar colecciones de datos (`tf.data.Dataset`). El modelo base corresponde a la metodología de “dos torres”, donde cada torre contiene:

- **user_id**: tensor con el ID de usuario convertido a entero.
- **product_id**: tensor con el ID de producto convertido a entero

Los identificadores de usuario y producto se encontraban en forma de cadenas de texto, por lo que se convirtieron a índices numéricos para previo a incorporarlos a las capas de *embedding*. Seguido, se realizó un mapeo de datos a la tupla (`input`, `label`), donde el *input* son las variables `user_id` y `product_id` y el *label* la variable objetivo. Posteriormente, entrenó en lotes de 256 ejemplos (para acelerar el entrenamiento y suavizar la estimación del gradiente) y en orden aleatorio (`.batch(256)` y `.shuffle(10000)`). Como consecuencia, se obtuvo un *pipeline* eficiente para la red neuronal, donde cada usuario y producto se codificaron como índices enteros compatibles con las capas de *embedding*.

La arquitectura base sigue el paradigma *two-tower* a través de:

- **Capas de *embedding***: Para cada usuario, la capa de *embedding* produce un vector denso de dimensión `embedding_dim` (32). A su vez, para cada producto, también existe una capa de *embedding* que produce un vector denso. Al final del entrenamiento, cada usuario y cada producto poseen un vector único de dimensión `embedding_dim`. Si un usuario y un producto son compatibles, sus vectores tienden a ser similares y al interactuar, el producto punto es alto. En cambio, si no existe relación entre ellos, el producto punto tiende a ser pequeño. Así, estos *embeddings* van aprendiendo el historial de interacciones pasadas y las correlaciones latentes entre grupos de usuarios y productos.

- **Cálculo del score:** se calcula un producto interno de la forma: `tf.reduce_sum(user_embeddings * product_embeddings, axis=1)`. Esto equivale al producto interno entre el vector de usuario y el de producto en el espacio latente. Se multiplica elemento a elemento y luego se realiza la suma mediante el `reduce_sum` para obtener un valor único por par usuario-producto. Cuanto mayor el producto interno, más fuerte la similitud entre usuario y producto.
- **Bias y MLP:** se utilizan en la versión optimizada, donde se agregan `user_bias` y `product_bias` como capas de *embedding* de dimensión 1 para capturar un sesgo intrínseco de cada usuario o producto. Además, se construyó una secuencia que recibe la concatenación entre usuarios y productos. Esta subred densa aprende relaciones no lineales más complejas que un simple producto interno. El resultado final score se ajusta con el *bias* de usuario y producto.

En resumen, cada torre se compone de un *embedding* que genera un vector latente. Esas torres convergen en una capa final para predecir la puntuación de relevancia.

Previo al entrenamiento del modelo se emplea Adam como optimizador. Esto agiliza la convergencia y maneja bien la dispersión de datos en recomendación. Para el modelo optimizado, se ajusta la tasa de aprendizaje mediante Keras Tuner.

Para evaluar el desempeño del modelo se divide el dataset en entrenamiento y validación y se realiza el cálculo de métricas de error como *RMSE* y métricas de clasificación como *Precision*, *Recall* y *F1-score*.

Una vez finalizado el modelo base, se implementó el Keras Tuner, que realiza una búsqueda automática de hiperparámetros para el modelo. El mismo generó combinaciones con distintos valores de cada hiperparámetro, como la dimensión de los *embeddings*, la cantidad de neuronas de la red neuronal densa (MLP), la tasa de aprendizaje, entre otros. Para cada configuración hiperparámetros, se construyó una instancia del modelo y la entrenó en un subconjunto de datos, evaluando la pérdida en validación. De esta forma,

explora de manera sistemática el espacio de combinaciones y selecciona la configuración con la menor pérdida promedio.

Reconociendo el desbalance original de la clase reordered, se replicó el mismo flujo de datos y la búsqueda de hiperparámetros sobre el dataset balanceado (mediante SMOTE) en el que se equipara la proporción de la variable reordered. En esta versión, la pérdida incorpora más ejemplos de la clase minoritaria mejorando la *Precision* pero reduciendo el *Recall*, como se muestra en el capítulo siguiente en la Tabla 7.5, reflejando el *trade-off* entre precisión y reproducibilidad.

En definitiva, la segunda etapa reaprovecha toda la infraestructura del modelo base con TFRS y la arquitectura “dos torres”, pero se enfoca en un conjunto de datos balanceado y en un nuevo ajuste de hiperparámetros específico a ese escenario.

7.3.2 Resultados

Los resultados numéricos de las cuatro variantes se resumen a continuación:

Conjunto de datos no balanceado:

Métrica	Baseline	Optimizado	Mejora (%)
RMSE	0.4476	0.3194	-28.64%
Precision	0.7851	0.8622	9.82%
Recall	0.6994	0.8949	27.95%
F1-score	0.7398	0.8783	18.72%

Tabla 7.4: Resultados de las métricas para el modelo base y optimizado con datos no balanceados.

Conjunto de datos balanceado:

Métrica	Baseline	Optimizado	Mejora (%)
RMSE	0.5768	0.4707	-18.39%
Precision	0.4089	0.4368	6.82%

Métrica	Baseline	Optimizado	Mejora (%)
Recall	0.3448	0.3167	-8.15%
F1-score	0.3741	0.3672	-1.84%

Tabla 7.5: Resultados de las métricas para el modelo base y optimizado con datos balanceados.

Mejor modelo balanceado vs mejor modelo no balanceado:

Métrica	No Balanceado	Balanceado	Mejora (%)
RMSE	0.3194	0.4707	47.37%
Precision	0.8622	0.4368	-49.34%
Recall	0.8949	0.3167	-64.61%
F1-score	0.8783	0.3672	-58.19%

Tabla 7.6: Comparativa de los mejores resultados de cada modelo.

El ajuste de hiperparámetros mejoró significativamente el desempeño de ambos modelos, especialmente en el modelo entrenado con el dataset sin balancear, donde *F1-score* aumentó de 0.7398 a 0.8783. El modelo que fue entrenado con los datos balanceados, introdujo ruido al entrenamiento, lo cual queda evidente en los resultados. Por otro lado, el modelo entrenado con el dataset balanceado no logró superar el rendimiento del modelo anterior. Esto sugiere que el proceso de remuestreo introdujo cierto grado de ruido que afectó negativamente la calidad de recomendación.

7.4 Surprise

Al igual que en los modelos previamente descritos, se parte de un modelo base que fue desarrollado utilizando los datos correspondientes al clúster 5. A partir de ese primer modelo, se observan oportunidades de mejora tanto en la distribución de las clases como en la configuración de los hiperparámetros; por lo cual se procede a realizar nuevos modelos con el mismo conjunto de datos, implementando *grid search* para la optimización

de hiperparámetros y ciertas modificaciones de features. En total se realizaron 4 modelos `Surprise` con el *dataset* correspondiente al clúster 5.

Como siguiente paso, se implementa una versión del modelo con datos balanceados, con el objetivo de reducir el sesgo de clase y realizar una evaluación más equitativa del rendimiento del sistema de recomendación. Utilizando este nuevo conjunto de datos se realizaron de igual manera 4 modelos siguiendo el mismo procedimiento previamente mencionado.

Finalmente se evaluaron las métricas obtenidas en cada ejecución para determinar cual modelo es el ganador.

7.4.1 Modelo Base

Como primer enfoque se implementó un modelo base que obra como punto de partida del resto de las pruebas realizadas. Este modelo se aplicó sobre un subconjunto de datos correspondiente a uno de los clústeres generados previamente (clúster 5).

Preparación de los datos:

Los datos utilizados contienen tres columnas clave: `user_id`, `product_id` y `reordered`. La variable `reordered` es binaria y representa si un usuario volvió a comprar un producto (1) o no (0). Debido a su naturaleza, se definió una escala de ratings de 0 a 1 al cargar el conjunto de datos en formato adecuado para `Surprise` mediante la clase `Reader`.

Entrenamiento del modelo:

Los datos fueron divididos en conjuntos de entrenamiento (80%) y prueba (20%) utilizando la función `train_test_split`; para luego, entrenar un modelo SVD con los siguientes parámetros iniciales:

- **`n_factors=30`**: número de factores latentes.
- **`lr_all=0.005`**: tasa de aprendizaje para todos los parámetros.
- **`reg_all=0.02`**: regularización para evitar sobreajuste.

Generación de recomendaciones:

Se generaron recomendaciones personalizadas para un usuario específico (`user_id = 0`). El sistema recorrió todos los productos disponibles y predijo la probabilidad de que el usuario vuelva a comprarlos. Luego, las recomendaciones fueron ordenadas por puntuación estimada y se realizó el mapeo a sus nombres reales para facilitar la interpretación.

7.4.2 Exploración de Variantes y Ajuste de Modelos

Con el objetivo de mejorar la calidad de las recomendaciones se desarrollaron y evaluaron varios modelos a partir del modelo base. Se exploraron mejoras mediante optimización de hiperparámetros, ingeniería de características y análisis de métricas.

Modelo Surprise2 – Búsqueda de hiperparámetros:

Este modelo replica el enfoque del modelo base, pero incorpora una búsqueda exhaustiva de hiperparámetros mediante la implementación de `GridSearchCV`:

Parámetros explorados:

- `n_factors=[20, 30, 50]`
- `lr_all=[0.002, 0.005, 0.01]`
- `reg_all=[0.02, 0.05, 0.1]`

Se seleccionaron los mejores valores según la métrica RMSE. El modelo entrenado con estos hiperparámetros logró una mejora tanto en RMSE como en las métricas de ranking.

Modelo Surprise3 – Validación y Robustez:

El modelo `Surprise3` repite el procedimiento de `Surprise2` pero refuerza la validación del modelo al mantener el mismo esquema de búsqueda y evaluación. Su objetivo fue verificar la consistencia del comportamiento del modelo optimizado y afinar detalles del pre procesamiento y salida de resultados.

Modelo Surprise4 – Ingeniería de Características:

En este modelo se redefine la variable de interacción, incorporando varias señales de comportamiento del usuario. Se realiza una visualización exploratoria de la distribución de esta nueva métrica, que permite capturar factores como el tiempo entre compras, el orden de agregado al carrito y la hora de compra. Esta nueva variable reemplaza a reordered como rating para el modelo SVD.

Los resultados arrojan un rendimiento mejorado en métricas como *Precision* y *F1-score*, que indican una mejora significativa en la calidad de las recomendaciones personalizadas, pero se observa que estas métricas alcanzan el mayor valor en el modelo base.

Resultados obtenidos con cada modelo:

Métrica	Surprise 1	Surprise 2	Surprise 3	Surprise 4
RMSE	0.4500	0.4492	0.4499	0.5629
Precision	0.6838	0.1680	0.1678	0.6834
Recall	0.7539	0.7376	0.7394	0.7663
F1-score	0.7171	0.2536	0.2535	0.6236

Tabla 7.7 Comparativa de los mejores resultados de cada modelo.

A partir del modelo base, se identificaron oportunidades de mejora que fueron abordadas en los modelos posteriores mediante optimización de hiperparámetros, validación cruzada y redefinición de la variable de *rating*. En base a este procedimiento se obtienen las siguientes conclusiones iniciales:

- El modelo Surprise2, que incorpora búsqueda de hiperparámetros con GridSearchCV, logró una leve mejora en RMSE; esto indica una menor desviación en las predicciones, pero por otro lado, en este modelo se redujeron significativamente los rendimientos en las métricas de ranking, *Precision* y *F1-score*.

- En el modelo Surprise4, se mejoró el *Recall* y si bien se ven mejoras en la métrica *F1-score*, el resultado se encuentra por debajo al obtenido en el modelo Surprise1.
- El modelo base (Surprise1) es el que presenta mejor equilibrio global entre todas las métricas, especialmente en *Precision* y *F1-score*, siendo además el único que logró destacar en ambos indicadores simultáneamente.

7.4.3 Balanceo de base de datos

Con el objetivo de reducir el sesgo de clase que existe en los datos originales, se implementó un procedimiento de balanceo utilizando la técnica SMOTE. Luego de aplicar esta técnica, se procedió a reentrenar y evaluar los cuatro modelos desarrollados previamente, utilizando esta nueva base. La principal motivación para realizar esta implementación, fue analizar cómo el balanceo afectaba el rendimiento de cada modelo, puntualmente en métricas de ranking como *Precision*, *Recall* y *F1-score*, ya que son sensibles a la distribución de clases.

Resultados obtenidos con cada modelo luego del balanceo de clases:

Métrica	Surprise 1	Surprise 2	Surprise 3	Surprise 4
RMSE	0.4229	0.4185	0.4190	0.4659
Precision	0.6960	0.0806	0.0809	0.7515
Recall	0.2142	0.4558	0.4578	0.8394
F1-score	0.3276	0.1252	0.1256	0.7422

Tabla 7.8 Comparativa de los mejores resultados de cada modelo con datos balanceados.

7.4.4 Resultados finales

Los resultados obtenidos luego de ejecutar los modelos con el conjunto de datos balanceado muestran variaciones significativas sobre las métricas arrojadas por los modelos ejecutados con el conjunto de datos inicial.

Métrica	Surprise 1 (Sin Balancear)	Surprise 4 (Balanceado)	Mejora (%)
RMSE	0.4500	0.4659	-3.53%
Precision	0.6838	0.7515	9.90%
Recall	0.7539	0.8394	11.34%
F1-score	0.7171	0.7422	3.50%

Tabla 7.9 Resultados de las métricas para el modelo base y optimizado con datos balanceados.

El modelo Surprise4 muestra una mejora considerable en *Precision*, *Recall* y *F1-score*, obteniendo los mejores valores entre todos los modelos evaluados. Esto muestra como la redefinición del rating, combinada con un conjunto de datos balanceado, permite capturar de manera más efectiva los patrones de recompra y generar recomendaciones altamente relevantes.

7.5 Otras Implementaciones

Previo a los modelos de recomendación detallados anteriormente, se exploraron dos enfoques basados en filtrado colaborativo que, si bien resultaron de utilidad, no llegaron a generar recomendaciones útiles para el conjunto de datos para recomendación.

En primer lugar se exploró la librería Implicit con su algoritmo ALS. Para este algoritmo, se tomó el conjunto de datos y se definió un *score* de interacción que incorpora el orden en el que se añaden los artículos al carrito y que tan reciente fue realizada la última compra. Se transformaron los identificadores de usuario y producto a índices enteros consecutivos y se añadieron como nuevas columnas al conjunto de datos. Con ello, se construyó una matriz dispersa de interacción usuario-producto necesaria para el cálculo de ALS. En un primer modelo se configuraron:

- 200 factores latentes.
- Regularización baja de 0.01.
- 100 iteraciones.

Esta elección de parámetros permite capturar matices finos sin sobre ajustar. Sin embargo, el modelo presentó errores de optimización que impidieron encontrar el balance entre construcción y regularización, y por lo tanto, no generó *embeddings* utilizables para ningún usuario.

En un segundo modelo, se creó utilizó *pivot table*, lo cual reestructura la tabla inicial generando la matriz de forma sencilla. Esta se dividió en sub conjuntos de entrenamiento y validación en un 80-20. Luego se ajustó ALS con la siguiente configuración:

- 50 factores latentes.
- Regularización de 0.1.
- 20 iteraciones.

Esta configuración busca una convergencia más estable en un espacio latente reducido. Aún así, los *scores* del modelo quedaron homogéneos por lo que el modelo no permitió discriminar productos relevantes. El espacio latente de este modelo no logró capturar patrones de preferencia lo suficientemente distintivos como para generar predicciones válidas.

Continuando con la exploración de enfoques de filtrado colaborativo, se implementó LightFM, con la finalidad de aprovechar su capacidad de combinar señales colaborativas y de contenido. El primer paso consistió en adaptar los datos, asignando índices únicos a usuarios y productos para crear una matriz de interacción dispersa en la que cada celda contenga el valor de *reordered*. A continuación, se generaron las características de usuarios y productos mediante la codificación *one-hot* y agregación por índice, convirtiendo estos atributos a una matriz de interacción para optimizar la eficiencia de acceso y cálculo.

Con esta estructura, se configuró un modelo de LightFM con:

- Pérdida *WARP*.
- 30 factores latentes.
- Tasa de aprendizaje de 0.05.

Esta configuración de parámetros busca maximizar la calidad de ranking de las primeras posiciones. De forma de acelerar la validación inicial se seleccionó un subconjunto reducido para el entrenamiento. Aún así, el modelo no logró ejecutarse, ya que dio varios errores que no se lograron resolver. Tras varios intentos de resolución, se priorizó la continuidad del proyecto y se decidió avanzar hacia enfoques alternativos, los cuales fueron descritos al inicio del presente capítulo.

8. Resultados

A continuación se ilustran los resultados obtenidos por modelo:

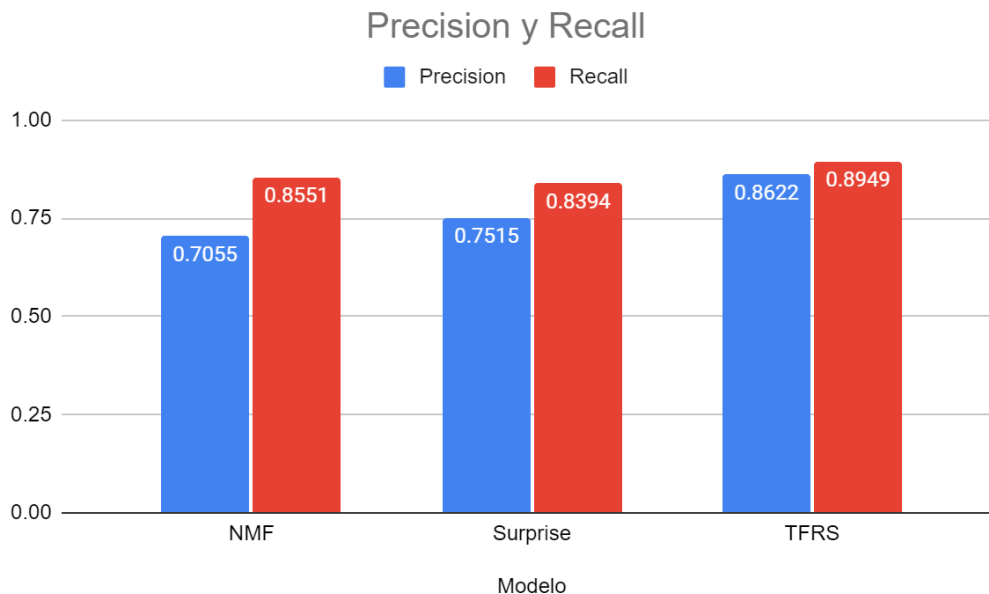


Figura 8.1: Gráfico de resultados de *Precision* y *Recall* para los tres modelos de recomendación.

En la Figura 8.1 se comparan las métricas de *Precision* y *Recall* obtenidas por los tres modelos evaluados: NMF, Surprise y TFRS. El modelo basado en NMF alcanzó un valor de *Recall* alto (0.8551), indicando gran capacidad para recomendar productos relevantes que efectivamente fueron comprados por los usuarios. Sin embargo, su *Precision* fue la más baja (0.7055), a pesar de estar en un rango de valores aceptable.

El modelo Surprise mostró un desempeño intermedio, con una *Precision* de 0.7515 y un *Recall* de 0.8394, reflejando mejor balance de *Recall* y *Precision*. Sin embargo, el modelo *two-towers* presentó el rendimiento más equilibrado, con una *Precision* de 0.8622 y un *Recall* de 0.8949. Indicando que no solo logra identificar los productos de mayor relevancia, sino que también minimiza la inclusión de productos irrelevantes en las recomendaciones.

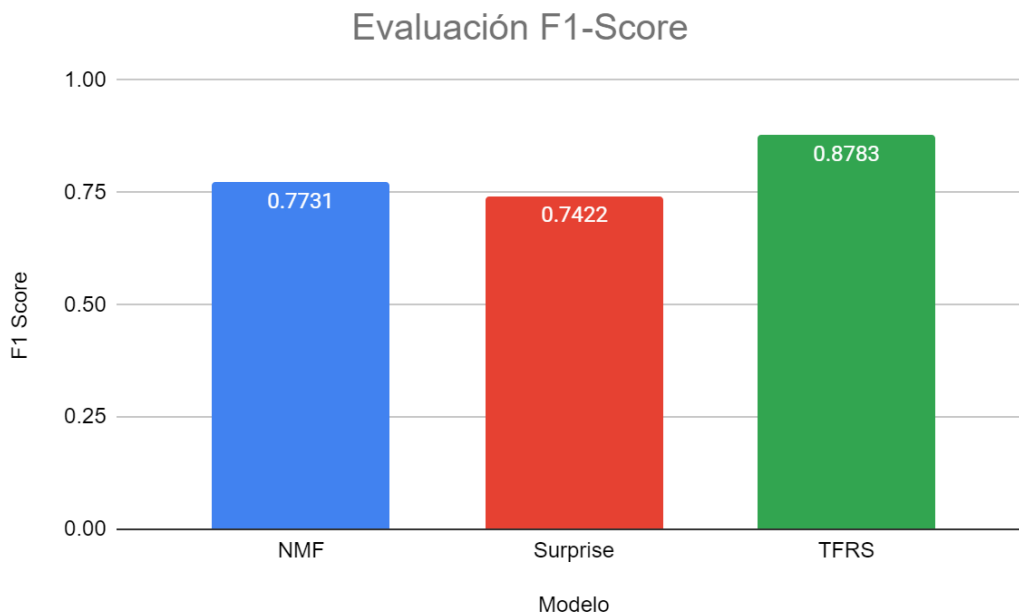


Figura 8.2: Gráfico de resultados de $F1$ -score para los tres modelos de recomendación.

Finalmente, al considerar el $F1$ -score como métrica de evaluación global, se observa con mayor claridad que el modelo *two-towers* alcanza el mejor desempeño, con un valor de 0.8783, seguido por NMF con 0.7731 y Surprise con 0.7422.

Este comportamiento puede explicarse por la naturaleza y capacidad de generalización de cada algoritmo. El modelo NMF, al tratarse de un método de descomposición matricial que se basa exclusivamente en las interacciones históricas entre usuarios y productos, logra capturar patrones latentes relevantes, esto le permite lograr un $F1$ -score competitivo. Sin embargo, su enfoque limitado a los datos de co-ocurrencia puede derivar en una menor personalización de las recomendaciones. Por otro lado, Surprise implementa técnicas de filtrado colaborativo con predicción explícita de preferencias, lo cual le permite modelar adecuadamente las relaciones entre usuarios y productos. Obteniendo valores similares a los del modelo NMF.

Por otro lado, el modelo *two-towers*, que integra embeddings tanto de usuarios como de productos, se beneficia de una representación contextual más rica. Esta arquitectura permite capturar interacciones más complejas y específicas, logrando un equilibrio óptimo entre precisión y cobertura. Esto se traduce en recomendaciones más personalizadas y

precisas, con un mejor balance entre precisión y cobertura. Por esta razón, el modelo *two-towers* supera claramente a las otras alternativas, consolidándose como la opción más efectiva para generar recomendaciones personalizadas en este caso de estudio.

9. Desafíos

El desarrollo del presente trabajo implicó la resolución de ciertos desafíos que fueron desde aspectos técnicos a decisiones metodológicas y conceptuales. A lo largo del proceso, fue necesario ir tomando decisiones importantes que nos permitieran asegurar tanto la calidad del análisis como el buen desempeño del sistema de recomendación construido.

9.1 Desafíos técnicos y de implementación

Preprocesamiento de Datos Masivos y exhaustivo proceso EDA

Uno de los primeros desafíos enfrentados estuvo relacionado con la manipulación y procesamiento de un conjunto de datos de gran volumen, compuesto por más de dos millones de registros.

Dada la estructura del dataset original, se debió realizar limpieza de datos, que incluyó la eliminación de registros nulos, la normalización de variables categóricas y la verificación de integridad en las distintas columnas. Sobre el tratamiento de valores nulos, el desafío estuvo centrado en la variable `days_since_prior_order`, la cual presentaba una proporción significativa de registros sin información. Ante esta situación, se tomó la decisión de eliminar estos registros ya que porcentualmente no afectaban al volumen del dataset (el total de valores nulos sobre el total de registros era de 6.2%). Esta elección se basó en la intención de ponderar la calidad de los datos y evitar así introducir “ruido” que pudiera afectar negativamente la etapa de entrenamiento del sistema de recomendación.

Transformación de Variables Categóricas con Alta Cardinalidad

Otro desafío se presentó durante la etapa de transformación de variables categóricas mediante la técnica de *One Hot Encoding*. Este procedimiento, fue necesario para adaptar los datos a los algoritmos utilizados. Esto implicó un riesgo concreto de explosión dimensional, especialmente en variables con alta cardinalidad como `product_name`, que contenía una gran cantidad de categorías diferentes.

La expansión del dataset a partir de la creación de nuevas columnas *dummy* impactó directamente en el tamaño del conjunto de datos y en los tiempos de procesamiento, aumentando la carga computacional en la etapa posterior de clusterización. Para intentar reducir estos efectos, se aplicó una estrategia de agregación por usuario.

Posteriormente a haber aplicado la técnica de *One Hot Encoding*, se agruparon las nuevas columnas por cada `user_id` único, sumando las cantidades de cada categoría para centralizar el comportamiento de compra de cada cliente en una única fila. Esta transformación permitió reducir significativamente la redundancia del dataset, manteniendo el total de la información de las variables categóricas y facilitando la aplicación de técnicas de clusterización y posterior recomendación.

Selección del Número Óptimo de Clústeres

En la etapa de segmentación de clientes mediante el uso de técnicas de clusterización, uno de los desafíos enfrentados fue la determinación del número óptimo de clústeres. Esta elección es muy importante, ya que afecta directamente a la conformación de los diferentes conjuntos de compradores y a la manera en que se concentran las variables dentro de cada grupo. Para tomar esta decisión, se realizaron gráficas de las diferentes variables categóricas para determinar cómo se distribuían las ocurrencias dentro de cada clúster generado. Para realizar este procedimiento se necesitó analizar cada ejecución con extremo cuidado y detalle, de manera de poder detectar qué número de clúster generaba agrupaciones de usuarios más diferenciados entre sí.

Problema de Cold Start

Un desafío identificado durante el desarrollo del sistema de recomendación fue el problema conocido como *cold start*, este fenómeno se presenta cuando se incorporan al sistema usuarios o productos nuevos que no cuentan con interacciones previas registradas en el conjunto de datos. Dado que los modelos de recomendación dependen de un historial de comportamiento para generar predicciones de calidad, la falta de información puede limitar la capacidad de recomendar productos relevantes.

En el presente trabajo, este inconveniente se vio en situaciones donde algunos usuarios presentaban muy pocas compras registradas, o en casos de productos que eran seleccionados por un número reducido de compradores.

Para resolver este problema, se implementó el uso de modelos híbridos que integran información de las compras y el análisis de clústeres. Esto permitió segmentar a los usuarios según comportamientos generales y generar recomendaciones basadas en el perfil del grupo al que pertenecen. Si bien el *cold start* no fue completamente eliminado, esta estrategia permitió reducir su impacto y mejorar el desempeño del sistema ante la presencia de este dilema.

9.2 Desafíos prácticos y académicos

Gestión de Herramientas y Librerías

Un aspecto importante del desarrollo fue la gestión y uso eficiente de las distintas herramientas y librerías utilizadas a lo largo del proyecto. El trabajo implicó combinar múltiples paquetes especializados, entre ellos Pandas para el manejo y transformación de datos, scikit-learn para tareas de preprocesamiento, clusterización y evaluación, Surprise para la implementación de modelos de recomendación basados en filtrado colaborativo, TensorFlow Recommenders para el desarrollo de modelos basados en aprendizaje profundo, entre otras librerías que fueron detalladas a lo largo del presente informe. Referente a este punto, el desafío se dio intentando lograr una integración eficiente entre estas herramientas, respetando las estructuras de datos requeridas por cada una y garantizando la compatibilidad entre las diferentes versiones de paquetes utilizados. Esta coordinación entre librerías, finalmente permitió construir un sistema de recomendación robusto y flexible, adaptado a las características del conjunto de datos trabajado.

Estudio Comparativo de Modelos de Recomendación y Selección de Métricas de Evaluación

Durante el desarrollo del sistema de recomendación, fue necesaria la lectura de documentación para comprender en profundidad el funcionamiento de los distintos modelos a implementar. El objetivo primordial, fue el de evaluar los diferentes rendimientos y determinar qué modelo se adaptaba mejor a las características del conjunto de datos y a los objetivos del proyecto. Se trabajó con modelos basados en filtrado colaborativo (SVD, NMF) utilizando la librería *Surprise*, así como con técnicas que consideraban interacciones implícitas mediante *Implicit*, y enfoques híbridos más avanzados implementados con TFRS.

Cada uno de estos modelos tiene particularidades en cuanto a cómo representa la relación usuario-producto, su capacidad de generalizar ante situaciones donde hay datos escasos y su comportamiento frente a problemas como el *cold-start* (comentado previamente). Por este motivo, fue necesario realizar un estudio cuidadoso del funcionamiento de cada sistema.

Finalmente, en cuanto a la evaluación del rendimiento de cada modelo, se decidió utilizar distintas métricas para medir y comparar los resultados. Se partió por el uso de RMSE para evaluar el error en predicciones, y *Precision*, *Recall* y *F1-score* para medir la calidad de las recomendaciones generadas. Si bien todas aportaron información útil, finalmente se optó por *F1-score* como métrica principal, ya que ofrece un equilibrio entre *Precision* y *Recall*, permitiendo así valorar de manera más integral la capacidad del sistema para recomendar ítems relevantes a los usuarios.

10. Conclusiones y Lecciones Aprendidas

Este proyecto comenzó con un análisis exploratorio y un proceso de *Feature Engineering* en los cuales se realizaron el preprocesamiento de los datos y la construcción de variables temporales, de frecuencia y de comportamiento de compra. A partir de estos datos previamente procesados, se aplicaron técnicas de segmentación de clientes como *K-means*, hasta definir una segmentación óptima con cinco grupos de clientes diferenciados. Esta etapa resultó clave para adaptar patrones de consumo homogéneos a cada modelo de recomendación, mejorando la personalización y reduciendo el ruido en entrenamientos posteriores.

Dado el caso de estudio, se analizaron métricas complementarias para evaluar el error de las predicciones (RMSE) y la calidad de las recomendaciones (*Precision* y *Recall*). Debido a la asimetría entre productos relevantes y no relevantes, se decidió considerar *F1-score* como métrica principal, ya que equilibra la proporción de aciertos y la capacidad de recomendación de productos relevantes, de forma de garantizar una comparación justa entre modelos .

Se implementaron y compararon tres enfoques de recomendación distintos:

- **NMF**, mostró su mejor versión al ser entrenado por el dataset sin balancear, logrando buenos resultados en cuanto a *F1-score*, evidenciando que al agregar datos sintéticos genera ruido y peores resultados, dada la naturaleza del modelo.
- **Surprise**, que a pesar de su facilidad de implementación, no tuvo el mejor desempeño en cuanto a *F1-score* en comparación con otros métodos.
- **TFRS**, con su arquitectura *two-towers* y aplicando *fine-tuning* ofreció el mejor equilibrio entre *Precision* y *Recall* al compararse con los demás modelos implementados.

Estos resultados demuestran que para maximizar la calidad de las recomendaciones fue de utilidad realizar una combinación entre segmentación previa, arquitectura profunda y búsqueda de hiperparámetros óptimos.

A lo largo del proyecto se demostró que la segmentación previa potenció la personalización al agrupar a los usuarios en perfiles homogéneos ya que redujo la varianza interna de los datos de entrenamiento y por ende aumentó la eficacia de cada modelo. Asimismo, dedicar recursos al fine-tuning de hiperparámetros permitió mejorar el desempeño del modelo.

Por otra parte, el uso de *SMOTE* para equilibrar las clases benefició al modelo Surprise pero en TFRS y NMF introdujo distorsiones que repercutieron en las métricas del algoritmo. Finalmente, al poner de manifiesto la idea de escalabilidad, y el manejo del cold-start hace necesaria la implementación de enfoques híbridos y de contar con una arquitectura web capaz de incorporar *feedback* en tiempo real.

En conjunto, los resultados cumplieron las expectativas del proyecto, se validó que una estrategia que combina una etapa de preprocesamiento, segmentación y modelos de recomendación ajustados al perfil del usuario logra recomendaciones precisas y relevantes. Por limitaciones de tiempo quedaron por explorar la integración del motor de recomendación en una aplicación *web* en producción, la incorporación de métricas de ranking avanzadas y la extensión a sistemas híbridos que combinen señales explícitas e implícitas en tiempo real. Estos desarrollos forman parte del camino a seguir para llevar el sistema de recomendación de un prototipo académico a una solución real.

11. Trabajo Futuro

Existen varias oportunidades de mejora y expansión que pueden ser implementadas en futuros desarrollos. Una de las acciones en las que se encuentra utilidad inmediata es la de la implementación del sistema de recomendación en una aplicación web, esto permitiría su despliegue a una audiencia más amplia y su integración con plataformas digitales.

11.1 Implementación en una Aplicación Web

Para llevar e implementar el sistema de recomendación en una aplicación web, se pueden considerar diferentes opciones como por ejemplo las que se enumeran a continuación:

- Desarrollo de una aplicación basada en frameworks como Django⁶ o Flask⁷ para gestionar las solicitudes de los usuarios y generar recomendaciones en tiempo real.
- Despliegue del sistema en servicios en la nube como AWS, Google Cloud o Azure, combinando el despliegue con el uso de contenedores con Docker y Kubernetes de manera de asegurar escalabilidad y disponibilidad.
- Otra alternativa, es utilizar una aplicación web como Streamlit [22]. Es un framework de código abierto que permite mostrar resultados en aplicaciones web interactivas sin necesidad de desarrollos complejos. Es sumamente útil para visualizar resultados de sistemas de recomendación, se puede desplegar en servicios como AWS o Google Cloud, y a su vez permite agregar visualizaciones con librerías como matplotlib o plotly para mejorar la presentación de los resultados.

La migración del sistema de recomendación a una aplicación web representa una evolución a lo desarrollado en el presente trabajo, ya que permite su aplicación en

⁶ Documentación Flask:
<https://flask-es.readthedocs.io/>

⁷ Documentación Django:
<https://docs.djangoproject.com/en/5.2/>

escenarios del mundo real. Se debe tener en cuenta que para implementar este proceso, se deben realizar nuevas investigaciones y pruebas, que aseguren el buen funcionamiento y una integración óptima con lo ya desarrollado. Ciertos aspectos importantes a tener en cuenta son los de incorporar mecanismos de seguridad y privacidad que garanticen la confianza a los usuarios finales.

11.1.2 Streamlit

El uso de `Streamlit` en el desarrollo y despliegue de sistemas de recomendación ofrece varios beneficios. Algunas de estas ventajas son:

Permite transformar scripts de *Python* en aplicaciones web interactivas. Además, no requiere contar con conocimientos avanzados en desarrollo web, lo cual reduce tiempos de implementación.

Tiene integración nativa con bibliotecas como `Pandas`, `NumPy`, `Scikit-learn`, `Matplotlib`, `Plotly` entre otras, lo que facilita la visualización de resultados, la exploración de datos y la presentación de métricas de evaluación del sistema de recomendación.

Permite crear interfaces donde los usuarios pueden ingresar sus preferencias y obtener recomendaciones en tiempo real, aportando una experiencia de usuario fluida y dinámica.

Instalación y Configuración de Streamlit:

La instalación de `Streamlit` es directa y puede realizarse dentro del entorno virtual del proyecto, lo cual es otro beneficio de la herramienta, ya que permite mantener organizadas las dependencias del desarrollo.

El primer paso para instalar la herramienta, es ingresar el comando `pip install streamlit`, mediante el uso del sistema de gestión de paquetes de *Python*. Esto descargará e instalará automáticamente la última versión estable de la herramienta desde el repositorio oficial de *Python*.

Luego de completarse la instalación, la documentación oficial de Streamlit recomienda ejecutar una aplicación de prueba incluida por defecto en la herramienta, esto permite verificar que la instalación fue exitosa y que el entorno se encuentra configurado de manera correcta. Esta verificación se realiza ingresando el comando `streamlit hello`, que ejecuta una interfaz de demostración en el navegador, confirmando la correcta comunicación entre el entorno de desarrollo y la aplicación web generada por Streamlit.

Con la herramienta ya instalada, el siguiente paso es avanzar en el desarrollo de una interfaz para el sistema de recomendación. Este proceso implica la creación de un archivo en el que se define la lógica de interacción con el usuario, la recolección de entradas y la presentación de las recomendaciones generadas por el modelo. Una vez que el archivo está definido, la aplicación se ejecuta con el comando `streamlit run "nombre_del_archivo.py"`, lo que lanza la interfaz gráfica en el navegador, permitiendo la interacción en tiempo real con el sistema [23].

12. Referencias Bibliográficas

- [1] Han, J., Kamber, M., & Pei, J. (2012). *Data Mining: Concepts and Techniques* (3.^a ed.). Oxford, Inglaterra: Morgan Kaufmann.
- [2] Rai, V., & Patre, P. (2019). *Combining DBSCAN and Grid Based Clustering for Performance Analysis*. Saarbrücken, Alemania: LAP Lambert Academic Publishing.
- [3] OpenAI. (2025). Imagen personalizada generada por IA del algoritmo DBSCAN, creada con DALL·E a través de ChatGPT [Imagen generada por IA]. ChatGPT. <https://chat.openai.com>
- [4] Falk, K. (2019). *Practical Recommender Systems*. Nueva York, NY: Manning Publications.
- [5] IBM. (s.f.). *Filtrado basado en contenido*. IBM. Recuperado de <https://www.ibm.com/es-es/think/topics/content-based-filtering>
- [6] IBM. (s.f.). *Filtrado colaborativo*. IBM. Recuperado de <https://www.ibm.com/es-es/think/topics/collaborative-filtering>
- [7] Recommender systems in machine learning – Examples. (n.d.). Vitalflux. Recuperado de <https://vitalflux.com/recommender-systems-in-machine-learning-examples/>
- [8] Naik, R. (Ed.). (s.f.). *Non-negative Matrix Factorization Techniques: Advances in Theory and Applications*. Berlín, Alemania: Springer.
- [9] Lee, D.D., & Seung, H. S. (2000). Algorithms for non-negative matrix factorization. *Advances in Neural Information Processing Systems*, 13, 556–562.
- [10] TensorFlow Recommenders. (s.f.). TensorFlow. Recuperado de <https://www.tensorflow.org/recommenders>

- [11] TensorFlow. (s.f.). *Introducing TensorFlow Recommenders*. Blog de TensorFlow. Recuperado de <https://blog.tensorflow.org/2020/09/introducing-tensorflow-recommenders.html>
- [12] Surprise Documentation. (s.f.). Surprise. Recuperado de <https://surprise.readthedocs.io>
- [13] Ferreira, L. (2024, enero 25). *Entendiendo la Biblioteca Surprise en Python para Sistemas de Recomendación*. Medium. Recuperado de https://medium.com/@lucas_fbr/entendiendo-la-biblioteca-surprise-en-python-para-sistemas-de-recomendación-bca70fdc32db
- [14] Implicit. (s.f.). Implicit. Recuperado de <https://implicit.readthedocs.io>
- [15] Frederickson, B. (s.f.). *Implicit: Fast Python Collaborative Filtering for Implicit Feedback Datasets*. GitHub. Recuperado de <https://github.com/benfred/implicit>
- [16] Hu, Y., Koren, Y., & Volinsky, C. (2008). *Collaborative Filtering for Implicit Feedback Datasets*. En *Proceedings of the 2008 Eighth IEEE International Conference on Data Mining* (pp. 263–272). Recuperado de <http://yifanhu.net/PUB/cf.pdf>
- [17] LightFM. (s.f.). Lyst. Recuperado de <https://making.lyst.com/lightfm/docs/home.html>
- [18] Prabowo, D. S. (2023, marzo 7). *Hybrid Recommendation System Using LightFM*. Medium. Recuperado de <https://medium.com/@dikosaktiprabowo/hybrid-recommendation-system-using-lightfm-e10dd6b42923>
- [19] Brownlee, J. (2017). *How to One Hot Encode Sequence Data in Python*. *Machine Learning Mastery*. Recuperado de <https://machinelearningmastery.com/how-to-one-hot-encode-sequence-data-in-python/>

[20] *KMeans clustering and PCA on wine dataset*. (2023, February 4). GeeksforGeeks. Recuperado de <https://www.geeksforgeeks.org/kmeans-clustering-and-pca-on-wine-dataset/>

[21] Scott, J., & Tũma, M. (2023). *Algorithms for sparse linear systems* (2023rd ed.). Birkhauser Verlag AG.

[22] Streamlit. (s.f.). Streamlit. Recuperado de <https://streamlit.io>

[23] Streamlit. (s.f.). Get started – Installation. Streamlit. Recuperado de <https://docs.streamlit.io/get-started/installation>

13. Glosario

Término	Definición
Adam	Adaptive Moment Estimation: Optimizador de descenso de gradiente adaptativo que ajusta la tasa de aprendizaje de cada parámetro con base en los momentos de primer y segundo orden.
Group by	Agrupación: Operación que consolida filas con la misma clave (p. ej. user_id) en un único registro con estadísticas agregadas.
ALS	Alternating Least Squares: Algoritmo de factorización matricial (especialmente para datos implícitos) que alterna la optimización de factores de usuarios e ítems.
AWS	Amazon Web Services: Plataforma de servicios en la nube de Amazon (computación, almacenamiento, bases de datos, etc.).
Azure	Microsoft Azure: Plataforma de nube de Microsoft para desplegar y escalar aplicaciones.
Batch	Batch: Conjunto de ejemplos procesados simultáneamente en una iteración de entrenamiento.
Bias Embedding	Bias Embedding: Vector de dimensión 1 que captura un sesgo (constante) por usuario o producto en un modelo de recomendación.
BPR	Bayesian Personalized Ranking: Función de pérdida que optimiza directamente el ordenamiento de ítems relevantes por encima de irrelevantes.
CBF	Content-Based Filtering: Paradigma que recomienda ítems comparando sus atributos con el perfil de intereses del usuario.
Centroides	Centroides: Promedios de las observaciones asignadas a cada clúster en K-Means; representan el “centro” de cada grupo.
CF	Collaborative Filtering: Paradigma de recomendación que predice preferencias a partir de interacciones de usuarios similares.

Término	Definición
Clusterización (Clustering)	Clusterización: Métodos no supervisados que agrupan observaciones por similitud para crear segmentos homogéneos.
Contenedores	Contenedores: Unidades autocontenidas (ej. Docker) que empaquetan una aplicación y sus dependencias para asegurar portabilidad.
CSR Matrix	Compressed Sparse Row: Formato de SciPy que almacena sólo los valores no nulos e índices de una matriz dispersa, ahorrando memoria.
CSV	Comma-Separated Values: Formato de texto donde los campos de cada registro están separados por comas.
DBSCAN	Density-Based Spatial Clustering of Applications with Noise: Algoritmo de clustering basado en densidad que agrupa puntos cercanos y marca como “ruido” los aislados.
Densidad de Ruido	Densidad de ruido: Conjunto de puntos que no cumplen la densidad mínima requerida y se etiquetan como ruido en DBSCAN.
Docker	Docker: Tecnología de contenedorización que empaqueta aplicaciones y dependencias en contenedores.
Dunn Index	Dunn Index: Cociente entre la distancia mínima inter-clúster y el diámetro máximo intra-clúster; valores altos indican buena separación y compacidad.
Duplicados	Duplicados: Registros idénticos en todas las columnas; se eliminan para evitar sesgos.
EDA	Exploratory Data Analysis: Conjunto de técnicas y visualizaciones para comprender la estructura, distribución y calidad de los datos antes del modelado.
Embedding	Embedding: Representación densa y de baja dimensión de un objeto discreto (usuario, producto) aprendida automáticamente para capturar similitudes.
Época (epoch)	Época: Un pase completo por todo el conjunto de entrenamiento durante el aprendizaje de un modelo.

Término	Definición
ETL	Extract, Transform & Load: Proceso de extracción, transformación y carga de datos desde múltiples fuentes a un repositorio objetivo.
Explained variance ratio	Explained Variance Ratio: Proporción de varianza explicada por cada componente en PCA.
F1-score	F1-score: Media armónica de Precision y Recall; balancea ambos aspectos en una sola métrica.
Feature Engineering	Feature Engineering: Transformación y creación de variables que resaltan patrones relevantes y mejoran el desempeño de los modelos.
Flask	Flask: Micro-framework de Python para desarrollar aplicaciones web ligeras y flexibles.
Google Cloud Platform	Google Cloud Platform: Suite de servicios en la nube de Google (VMs, bases de datos, ML, etc.).
Grid Search / GridSearchCV	Grid Search / GridSearchCV: Estrategia de búsqueda exhaustiva de hiperparámetros evaluando todas las combinaciones de un espacio predefinido.
HDBSCAN	Hierarchical Density-Based Spatial Clustering of Applications with Noise: Extensión jerárquica de DBSCAN que produce clústeres estables sin fijar un único parámetro de densidad.
Hiperparámetro	Hiperparámetro: Parámetro fijado antes del entrenamiento que no se ajusta por gradiente.
Implicit (biblioteca)	Implicit: Librería de Python para filtrado colaborativo con señales implícitas; implementa ALS, BPR, etc.
Índice Calinski-Harabasz	Calinski-Harabasz Index: Razón entre dispersión inter-clúster e intra-clúster; valores grandes implican buena separación y compacidad.
Inercia (K-Means)	Inercia: Suma de las distancias cuadráticas de cada punto a su centroide en K-Means; mide la compacidad del clúster.
Inicialización NNDSVDa	NNDSVDa: Técnica de inicialización para NMF basada en SVD que garantiza matrices no negativas y mejora la convergencia.

Término	Definición
Keras Tuner	Keras Tuner: Herramienta que automatiza la búsqueda de hiperparámetros en modelos TensorFlow/Keras.
K-Means	K-Means: Algoritmo que particiona los datos en K grupos minimizando la varianza interna.
Kubernetes	Kubernetes: Orquestador de contenedores que automatiza despliegue, escalado y gestión de aplicaciones.
LightFM	LightFM: Librería de Python para sistemas de recomendación híbridos (WARP, BPR, log-loss).
MAE	Mean Absolute Error: Promedio de las diferencias absolutas entre predicciones y valores reales.
Marco Teórico	Marco Teórico: Contexto conceptual que sustenta la investigación; se detalla en el Cap. 2 de la monografía.
Matriz de Correlación de Spearman	Matriz de Correlación de Spearman: Tabla de coeficientes de correlación de rango entre pares de variables; detecta relaciones monótonas.
Método del Codo	Método del Codo: Gráfica de la inercia en K-Means para elegir el número óptimo de clústeres en el punto donde la curva deja de mejorar significativamente.
MLP	Multi-Layer Perceptron: Red neuronal densa con múltiples capas ocultas y activaciones no lineales.
MSE	Mean Squared Error: Promedio de los cuadrados de las diferencias entre predicciones y valores reales.
Muestra Estratificada	Muestra Estratificada: Sub-muestra que preserva las proporciones originales de cada clase o clúster.
NMF	Non-negative Matrix Factorization: Factorización matricial que descompone una matriz en dos no negativas, revelando factores latentes interpretables.
One Hot Encoding	One Hot Encoding: Transformación que convierte cada categoría en una columna binaria (0/1).
PCA	Principal Component Analysis: Reducción de dimensionalidad que proyecta los datos a componentes ortogonales ordenadas por varianza explicada.

Término	Definición
Pivot table / pivot_table	Pivot table / pivot_table: Reestructuración de un DataFrame para transformar valores de una columna en columnas de una matriz.
Precision / Precision@K	Precision / Precision@K: Proporción de ítems recomendados que son relevantes; @K evalúa los K primeros puestos.
Privacidad	Privacidad: Conjunto de políticas y técnicas para proteger datos personales y derechos de los usuarios.
Producto interno	Producto interno: Operación que calcula la similitud entre dos vectores vía suma de productos elemento a elemento.
Reader (Surprise)	Reader (Surprise): Clase que define formato y escala de ratings al convertir un DataFrame en Dataset de Surprise.
Recall / Recall@K	Recall / Recall@K: Proporción de ítems relevantes que el modelo recupera; @K restringe al top K.
RMSE	Root Mean Squared Error: Raíz del Error Cuadrático Medio; penaliza fuertemente los errores grandes.
Score de interacción	Score de interacción: Ponderación numérica (basada en reorden, recencia, etc.) que indica la fuerza de cada interacción usuario-producto.
Segmentación horaria	Segmentación horaria: Agrupación de horas en Morning (6-12), Afternoon (13-17), Night (18-22) y Dawn (23-5).
Segmentación por rango de órdenes	Segmentación por rango de órdenes: Clasificación de usuarios en grupos según escalas de max_order.
Silhouette Score	Silhouette Score: Métrica (-1 a 1) que evalúa qué tan bien está cada punto separado de otros clústeres respecto a su propio clúster.
SMOTE	Synthetic Minority Over-sampling Technique: Sobremuestreo sintético de la clase minoritaria mediante interpolación entre vecinos próximos.
SVD	Singular Value Decomposition: Descomposición matricial en tres matrices que revela factores latentes; base de muchos métodos de recomendación.

Término	Definición
tf.data.Dataset	tf.data.Dataset: API de TensorFlow para construir tuberías de datos eficientes (lectura, shuffle, batch).
train_test_split	train_test_split: Función que divide un conjunto de datos en subconjuntos de entrenamiento y prueba.
Visualización Univariante	Visualización Univariante (Histogramas): Gráficos de barras que describen la distribución de una sola variable; se colorean por clúster para comparar grupos.
WARP	Weighted Approximate-Rank Pairwise: Pérdida que optimiza directamente la calidad de ranking en feedback implícito, priorizando ítems relevantes en primeras posiciones.

Anexo

Anexo 1: Estructura del repositorio en *GitHub*

Para facilitar la reproducción del proyecto, el conjunto de datos, el código en notebooks ejecutados y en *scripts* de *Python* se encuentran en el siguiente repositorio público de GitHub

A continuación se describe la estructura que sigue el repositorio y su contenido.

El contenido del repositorio se divide en cinco bloques principales:

00_Data_Bases

Contiene todos los conjuntos de datos trabajados, donde el archivo base es `Supermercado.zip`

01_Exploratory_Analysis

Este bloque contiene dos documentos en formato *Jupyter Notebook*. El primero se llama `EDA.ipynb` en el que se realizó la carga de datos inicial, limpieza, estadística descriptiva y visualizaciones y el segundo es `Feature_Engineering_One_Hot.ipynb` en el que se realizaron las modificaciones descritas en el Capítulo 4 del presente trabajo.

02_Modeling

Contiene dos *Jupyter Notebooks*, el primero es `Clustering_KMeans_DF_OHE.ipynb`, en el que se realiza la segmentación aplicando *K-means* y luego se encuentra `DS_Balancing.ipynb` en el que se realiza el balanceo de datos del conjunto de datos que resulta del *notebook* anterior.

03_Recommendation_System

En este apartado se implementaron los algoritmos de recomendación. En el mismo se encuentran tres carpetas principales: *NMF*, *Surprise* y *Tensor_Flow_Recommender*, cada una de ellas contiene los documentos en *Script* de *Python* en los que se realizaron los sistemas de recomendación, organizados en carpetas según se haya ejecutado con el

conjunto de datos balanceado o desbalanceado. A su vez, se encuentran archivos en formato *.html* en el que se resumen y explican los resultados de cada ejecución.

04_Testing

En el cual se encuentran todas las versiones intermedias o de prueba que no otorgaron resultados esperados o que fueron descartadas de la versión final. Cada una en su respectiva carpeta: *Exploratory_Analysis*, *Modeling* y *RecommendationSystem*.

.gitignore

Aquí se especifican patrones de archivos y carpetas excluidos del control de versiones